

Hyperon for AGI $\Rightarrow$ ASI

Whitepaper 2025

Deepish-Dive Version

Ben Goertzel

November 29, 2025

Abstract

This whitepaper presents the Hyperon AGI platform and coupled PRIMUS cognitive architecture as a unified neurosymbolic framework designed to progress from here to AGI to beneficial ASI.

At Hyperon’s core software-wise is Atomspace, a typed, content-addressed metagraph that co-locates symbols, tensors, truth values, motives, and edit operations, enabling distinct cognitive processes to interoperate over one memory and control plane. Atomspaces can be implemented on multiple fabrics, one interesting substrate is MORK, a prefix-tree based metagraph database centered around a robust and scalable path algebra. Neural networks are integrated via two complementary neural-integration modes: (i) Outside integration that wraps existing models as Spaces to expose embeddings and activations for reasoning; and (ii) Inside integration via methods such as QuantiMORK, which encodes wavelet-structured tensors directly in the PathMap and supports predictive-coding style local updates.

The PRIMUS cognitive layer orchestrates goal-directed and ambient loops atop networks of Atomspaces using a diversity of cognitive mechanisms bound together by a small set of mathematical controls: quantale-based weakness (a system-wide simplicity prior) and geodesic effort (even-cost forward/backward control). Recent advances include WILLIAM-on-MORK for adaptive compression and attention steering, PLN on quantale-annotated factor graphs, SubRep for certificate-driven subgoal/option admission (CDS/PDS), MetaMo for compositional, auditable motivation, TransWeave for algebraically governed transfer and composition across tasks and paradigms, and a ”bridging” approach for finding models balancing accuracy with simplicity via following geodesics from very simple to very accurate models.

Long-standing AGI algorithms like PLN for uncertain inference, MOSES for probabilistic evolutionary program learning and ECAN for self-organizing attention allocation are given new dynamics and new tools using these new approaches.

The stack compiles the MeTTa language into a variety of different underlying targets, including MeTTa-IL kernels running efficiently on MORK; and supports decentralized, capability-secured execution (RSpace/Rholang, F1R3FLY/ASI-Chain) with content-addressed provenance, shadow $\rightarrow$ elevate $\rightarrow$ rollback deployment, and reproducible twins. A specialized MM2 language allows hyper-efficient implementation of critical subsystems directly on MORK.

As examples of the diverse avenues for practical applicability, we outline pilots in games, social robotics, bioinformatics, and mathematics; and also a research programme to study the properties of the framework in terms of efficiency, stability, transfer, and benefit.

Overall, what we describe is an OS-for-cognition that couples scalable transparent reasoning and learning via multiple synergetic methods, integrated neural dynamics, typed self-modification, and decentralized governance into a single, auditable path from AGI to beneficial ASI.

Contents

1	Executive Summary	6
1.1	Core Vision	6
1.2	Technical Architecture	7
1.3	Neural Network Integration: Two Complementary Approaches	7
1.3.1	Outside Mode: Pragmatic Integration	7
1.3.2	Inside Mode: Deep Unification with QuantiMORK	7
1.4	PRIMUS: The Cognitive Architecture	8
1.4.1	PRIMUS Technical Implementation	9
1.5	Key Advances Since 2023	9
1.6	From AGI to ASI	10
1.6.1	Goal Stability Under Self-Modification	10
1.6.2	Shared Algebraic and Dynamical Controls	10
1.6.3	Decentralized Infrastructure and Governance	10
2	Hyperon System Design	10
2.1	The Technical Foundation	11
2.2	The Space API: A Universal Interface	11
2.2.1	How Spaces Work	12
2.3	MORK: The High-Performance Core	12
2.3.1	Weighted Atom Sweeps: Cognitive Scheduling	12
2.3.2	Executing Dense Kernels: ShardZipper and ByteFlow	13
2.4	Neural Network Integration	13
2.4.1	The QuantiMORK Advantage	14
2.5	Distributed Execution and Scaling	14
2.5.1	Decentralized Execution	14
2.6	State Management and Persistence	15
2.7	Putting It All Together	15
3	The Language Stack	15
3.1	A Multi-Layer Approach to Cognitive Programming	15
3.2	The Technical Stack	16
3.3	MeTTa: Cognitive Code as Graph Transformations	17
3.3.1	The Execution Model	17
3.4	MeTTa-IL: The Compilation Bridge	17
3.4.1	The GSLT Foundation	18
3.5	PeTTa and JeTTa	18
3.6	MORKL/MM2: Where Hot Loops Live	18
3.6.1	Core Operations	19
3.7	PyMeTTa: Developer Ergonomics	19
3.7.1	Clean Transpilation	19
3.7.2	Performance Integration	20
3.7.3	LLM-Assisted Development	20
3.8	Distribution and State Management: Handling Multi-Node Cognition	20
3.9	Sketch of the Development Workflow	20

4	The PRIMUS Cognitive Architecture	21
4.1	Two meta-dynamics: goal-directed and ambient cognition.	21
4.2	How the pieces cooperate inside those loops.	22
4.3	Why this composes and scales on Hyperon.	22
4.4	Long-time Hyperon components in this picture	22
4.5	Newly introduced ingredients that make PRIMUS even more of a coherent “mind.” .	23
4.6	Shared controls unify the two loops.	23
5	Major Additions to PRIMUS Since 2023	23
5.1	Weakness Theory: The Mathematics of Simplicity	24
5.1.1	The PLN-Quantale Weakness Framework	24
5.1.2	Neural Network Applications	25
5.2	Geodesic Inference and Control	25
5.2.1	Mathematical Foundation	25
5.3	Fluid Dynamics for Attention: ECAN Meets Optimal Transport	26
5.3.1	Preserving ECAN Semantics with Enhanced Transport	26
5.3.2	Implementation on MORK via ShardZipper	26
5.4	Incompressible-Fluid Networks	26
5.4.1	Core Mathematical Framework	27
5.5	Integration with Predictive Coding	27
5.6	Connections to Transweave and Neural-Symbolic Integration	27
5.7	Schrödinger Bridge Learning: From Abstract to Detailed Models	28
5.7.1	Integration with Predictive Coding Networks	28
5.7.2	Evolutionary Programming Applications	28
5.7.3	Conditions for Effectiveness	28
5.8	TransWeave: Intelligence Through Intertwining	29
5.8.1	Mathematical Foundations	29
5.8.2	Impossibility Results and Selective Transfer	29
5.8.3	Integration with Geodesic Control	29
5.8.4	Commutativity FTW	30
5.9	SubRep: Integrating Subgoal Decomposition and Representation Learning	30
5.10	MetaMo: A Compositional Motivation Architecture	31
5.10.1	The Mathematics of Motivation	32
5.10.2	Operational Design	32
5.10.3	A Step Beyond OpenPsi and other Earlier Motivational Frameworks	33
5.11	Algorithmic Chemistry: Computation via Reaction Networks	33
5.11.1	Technical Architecture	33
5.11.2	Integration with PRIMUS	33
5.11.3	Why Hyperon?	34
5.12	WILLIAM-on-MORK: Efficient Adaptive Compression for General Cognition	34
5.12.1	Technical Implementation	34
5.12.2	Neural Network Acceleration	35
5.13	How These Advances Work Together	35
5.14	The Path Forward	36

6	Re-Visioning of Longstanding PRIMUS Components	36
6.1	PLN (Probabilistic Logic Networks).	37
6.1.1	PLN Control via Quantale-Annotated Factor Graphs	37
6.2	Evolutionary Program Search	38
6.2.1	GEO-EVO: Evolutionary Program Search	38
6.3	ECAN (Economic Attention Network).	38
6.4	Pattern mining on general graphs.	39
6.5	Concept blending.	39
7	Neural-Symbolic Synergy in Hyperon	39
7.1	Two Paths to Integration	40
7.2	Symbolic Heads: Giving Transformers Structured Memory	40
7.2.1	The Problem with Pure Attention	40
7.2.2	How Symbolic Heads Work	40
7.2.3	Where the Templates Come From	41
7.3	Pattern Mining as Neural Prior Discovery	41
7.4	Inside Atomspace: The (More Radical) QuantiMORK Approach	42
7.4.1	Neural Networks as Native Graph Operations	42
7.4.2	The Technical Architecture	42
7.4.3	Predictive Coding Without Backprop	42
7.5	Weakness-Based Stability for Neural Updates	42
7.5.1	The Challenge of Compositional Learning	42
7.5.2	Commutativity as a Design Principle	43
7.5.3	Connection to MetaMo	43
7.6	WILLIAM-Guided Neural Efficiency	43
7.6.1	Compression Drives Computation	43
7.6.2	Dynamic Sparsity in Practice	43
7.7	Example Neural/Hyperon Workflows	44
7.7.1	Workflow A: Outside Integration with Symbolic Augmentation	44
7.7.2	Workflow B: Inside Integration with QuantiMORK	44
7.8	The Deeper Significance	45
8	Planning for the AGI $\rightarrow$ ASI Transition: Stability, Reflection, and Safe Self-Modification	45
8.1	The Challenge of Ongoing Radical Self-Improvement	45
8.1.1	The Technical Framework	46
8.2	Goal Stability as a Tractable Math Problem	46
8.2.1	The Mathematics of Stability	46
8.3	Global Regulators: Consistent Principles Across All Cognition	47
8.4	The Self-Modification Pipeline	47
8.4.1	Beyond Hot-Patching	47
8.4.2	Technical Details	47
8.5	Safety Through Transparency	48
8.5.1	Making Safety Checkable	48
8.6	Decentralized Governance	48
8.6.1	Multi-Party Evolution	48
8.7	A Concrete Example: Upgrading Predictive Coding	49
8.7.1	Walking Through the Pipeline	49

8.8	Implementation Checklist	49
8.8.1	Building Self-Improvement Capabilities	49
8.9	The Core Insight	50
8.9.1	Unity of Principles	50
9	Example Application Domains: From Current Pilots to General Intelligence	50
9.1	Game AI: From Minecraft to Sophiaverse and the Neoterics Playground	51
9.1.1	The Opportunity	51
9.1.2	Technical Implementation	51
9.2	Humanoid Social Robotics: Education and Performance	52
9.2.1	The Partners and Vision	52
9.2.2	Technical Architecture	52
9.3	Bioinformatics: Graph-Based Discovery for Longevity and Beyond	53
9.3.1	Why Hyperon Excels Here	53
9.3.2	System Architecture	53
9.4	Mathematical Theorem Proving with Automated Conjecturing	54
9.4.1	Beyond Proof to Discovery	54
9.4.2	Technical Design	54
9.5	Why These Domains Matter Together	55
9.5.1	Shared Infrastructure Wins	55
10	Why This Is a Path to Beneficial AGI	55
10.1	Reflective Motivation and Self-Modification	55
10.1.1	The Gap in Current Systems	55
10.1.2	How It Works Technically	56
10.1.3	Why This Promotes Beneficial Behavior	56
10.2	Decentralized Implementation on F1R3FLY/ASI-Chain	56
10.2.1	The Governance Problem	56
10.2.2	The Technical Solution	57
10.2.3	Why Decentralization Enables Benefit	57
10.3	Beneficial Applications as Training Ground	57
10.3.1	Shaping Through Practice	57
10.3.2	Domain-Specific Implementation	57
10.3.3	Why These Applications Matter	58
10.4	The Self-Reinforcing Loop	58
10.4.1	Concrete Safeguards	58
10.4.2	Measurable Success	59
10.5	Addressing Common Concerns	59
10.5.1	The Bottom Line	59
11	Summary & Conclusion: A Viable, Faster Path to Beneficial AGI	60
11.1	Why This Path Is Plausible	60
11.2	The Technical Through-Line	60
11.2.1	One Substrate for Cooperative Components	60
11.2.2	Unified Biases and Control Laws	60
11.2.3	First-Class Reflection and Self-Modification	60
11.2.4	Flexible Neural Integration	61
11.2.5	Decentralized, Verifiable Deployment	61

11.2.6	Beneficial Applications as Training Data	61
11.3	Why This Can Be Faster to Human-Level AGI	61
11.3.1	Computational Efficiency	61
11.3.2	Cumulative Learning	61
11.3.3	Reduced Technical Debt	62
11.4	Why This Leads to Beneficial Superintelligence	62
11.4.1	Aligned Growth Dynamics	62
11.4.2	Democratic Pressure by Design	62
11.4.3	Positive-Sum Development Environment	62
11.5	Falsifiable Near-Term Predictions	62
11.6	Final Thoughts	63

1 Executive Summary

1.1 Core Vision

Hyperon fundamentally reimagines how an AGI system should be built. While AGI is now increasingly recognized as a viable scientific and engineering goal, most approaches focus on:

- Just training a big deep neural net with an (hopefully) appropriate architecture and then filling in its gaps by having it access a grab-bag of external tooling; or else
- For those few not doing (1): Stitching together a hybrid system from separate subsystems for neural discrimination/generation, structured knowledge storage/retrieval, symbolic reasoning and so forth

Hyperon takes a quite different approach, and instead provides a unified, integrated software and conceptual foundation wherein multiple AI processes carrying out different forms of computational and cognitive processes can live together and cooperate and synergize on a fundamental level.

Think of it as an operating system designed specifically for cognition, where concepts, facts, goals, procedures, and even neural network components all exist as first-class citizens in a shared knowledge metagraph. Unlike a standard computer operating system where part of the job of the OS is to keep different processes out of each other's way, though, a cognitive operating system needs to do something subtler: It needs to help the different cognitive processes that are running concurrently to interoperate appropriately ... to stay out of each others way when that's the best thing, or to help each other out when this is what's needed, sharing the information needed to make this help viable.

This unified, integrative approach solves a critical problem in AGI development:

- The inability of any one approach to intelligence (pattern recognition, reasoning, evolution...) to solve on its own ALL the issues involved in manifesting AGI at the human level or beyond
- The friction and inefficiency that comes from forcing different types of intelligence to communicate through narrow and rigid interfaces.

Logic and neural modeling/synthesis and evolution, for instance, are each excellent solutions to different aspects of the human-like-AGI problem. However fitting these different approaches together in a naive way leads to numerous issues. When logical reasoning needs to inform neural network training, or when pattern recognition should guide symbolic proof search, traditional architectures make this difficult and slow. Hyperon eliminates these barriers by sophisticated mathematics-driven and computer-architecture-savvy design.

1.2 Technical Architecture

At its heart, Hyperon is built around the Atomspace, a typed, content-addressed, flexibly traversable metagraph that serves as the universal substrate for all cognitive activity. Every piece of knowledge—whether it’s a fact, a procedure, a neural network node or link, or a control signal—is represented as an “Atom” that other components can directly access and manipulate.

Atomspaces may be manifested on the back end in a variety of ways; we will pay particular attention here to one such manifestation, which is MORK, a high-performance storage and compute engine that seems to be providing particular payoff for a variety of Hyperon AI algorithms when used as an Atomspace backend. MORK provides several critical capabilities:

- **PathMap/Merkle-DAG structure:** Enables near-constant time lookups of related information
- **Lock-free operations:** Multiple cognitive processes can run simultaneously without blocking each other
- **Unified memory layouts:** Both GPU-accelerated neural operations and symbolic reasoning can operate on the same in-memory structures

To scale beyond a single machine, DAS (Distributed Atomspace) extends this architecture across clusters, while specialized Spaces provide focused execution environments—including Neural Spaces for deep learning models and “MettaCycle” infrastructure encapsulating RSpace/Rholang hosts for secure, capability-controlled execution on multi-core hardware and across centralized or decentralized networks.

1.3 Neural Network Integration: Two Complementary Approaches

One of Hyperon’s key innovations is how it handles neural networks. Most neural-symbolic AI systems treat neural models as black boxes that sit outside the main cognitive architecture, communicating only through limited APIs. This separation creates fundamental limitations: neural insights can’t easily inform symbolic reasoning, and symbolic structures can’t efficiently guide neural computation.

Hyperon breaks new ground by offering two approaches to neural integration, each suited to different needs:

1.3.1 Outside Mode: Pragmatic Integration

In “outside” mode, existing neural networks (large language models, vision systems, specialized networks) run as external services but are wrapped to appear as native Hyperon Spaces. Their outputs—embeddings, predictions, attention patterns—become Atoms that symbolic reasoning can directly manipulate. This approach offers immediate compatibility with existing models and frameworks while still enabling meaningful integration.

1.3.2 Inside Mode: Deep Unification with QuantiMORK

The “inside” mode represents a more radical integration. Through the QuantiMORK system (based on prior work with “waveformers”), neural network tensors are encoded as multiresolution directed acyclic graphs directly within MORK. This means:

- Tensor operations happen on the same PathMap structures that store symbolic knowledge

- Symbolic rules can directly orchestrate neural computations, deciding when and where to refine predictions
- Pattern mining can identify important features that become attention heads in real-time
- The system can selectively expand computation only where uncertainty is high, dramatically improving efficiency

Ground-level operations like convolution, attention, and matrix multiplication run on PathMap-addressed subtrees, while MeTTa rules schedule forward passes and predictive coding updates. This tight integration enables true neurosymbolic synergy—symbolic and neural processes share not just data, but the actual computational substrate.

1.4 PRIMUS: The Cognitive Architecture

PRIMUS is Hyeron’s “mind layer”: a neurosymbolic cognitive architecture that runs two interleaved loops over one shared metagraph memory (Atomspace). A goal-directed loop uses MetaMo to maintain and negotiate top-level motives, then composes procedures—drawing on PLN uncertain reasoning, MOSES/GEO-EVO evolutionarily-learned skills, predictive-coding-based neural models, and options certified by SubRep goal-refinement. Methods, etc.—to pursue goals in context. An ambient loop, scheduled by ECAN nonlinear-dynamical attention allocation, continuously mines patterns, blends concepts, and tightens beliefs with factor-graph-based PLN reasoning so the system keeps discovering reusable structure even off-task.

TransWeave provides the algebra that lets these processes reinforce each other and generalize across tasks (transfer/compose with bounded order effects), while quantale-based weakness offers a system-wide simplicity bias.

All of this is expressed as MeTTa graph rewrites and executed efficiently on MORK (content-addressed PathMap, Weighted Atom Sweeps) and across the Distributed Atomspace, so PRIMUS scales from a single machine to decentralized clusters while staying auditable, modular, and composable end-to-end.

These particulars illustrate how, as we have said earlier, rather than choosing a single approach to intelligence, PRIMUS orchestrates multiple complementary methods in a unified cognitive cycle:

- **Perception and Action:** Predictive coding networks process sensory input and generate motor commands
- **Attention and Memory:** An ECAN (Economic Attention Network) system manages what’s important, using economic principles to allocate cognitive resources
- **Reasoning:** Probabilistic Logic Networks (PLN) handles uncertain inference through geodesic-guided forward and backward chaining, and factor graph message passing
- **Learning:** Multiple parallel systems extract patterns, e.g.:
 - WILLIAM mines reusable motifs and programs using compression principles
 - MOSES performs evolutionary program search guided by goals and utility estimates
- **Control:** Geodesic inference finds the shortest-effort paths between current states and goals
- **Motivation:** The MetaMo system manages goal hierarchies and resource allocation

The key point is: These aren’t isolated modules—they share common data structures, evaluation metrics, and control signals (assisted by a common implementation in terms of quantale weakness, geodesic control, and TransWeave based mapping across tasks, methodologies and representations). When WILLIAM discovers a useful pattern, it immediately becomes available to PLN for reasoning and to the neural networks as an attention target. When PLN identifies an important inference gap, it can direct MOSES to evolve programs to fill it. Etc.

1.4.1 PRIMUS Technical Implementation

Each PRIMUS component is implemented to maximize both individual effectiveness and system-wide integration; as a few illustrative examples:

- PLN operates through quantale-annotated factor graphs, enabling parallel message passing that carries both logical structure and uncertainty measures
- WILLIAM uses minimum description length principles to identify patterns worth remembering
- MOSES maintains populations of program variants, using estimation of distribution algorithms (EDA) to efficiently explore program space
- Geodesic control treats cognition as an optimal transport problem, balancing forward reachability with backward usefulness
- MetaMo uses a formal subrepresentation framework to decompose abstract goals into concrete, achievable subgoals

1.5 Key Advances Since 2023

Since the last technical whitepaper for Hyperon was written in 2023, Hyperon has evolved in several significant ways:

- **Launch of a scalable infrastructure.** The MORK storage/compute engine provides a high-performance underpinning of Atomspace, and the integration of MORK into the MetaCycle infrastructure gives a high-performance concurrent and decentralized processing infrastructure
- **Adaptation of MeTTa language to MORK/rholang infrastructure.** We have gained considerable experience making prototype implementations of PRIMUS algorithms in MeTTa, and have also understood ways in which the initial variants of MeTTa needed to be adjusted for optimal performance on the scalable infrastructure we have developed
- **Introduction of a number of additional AGI algorithms** into the picture, fitting within the overall framework of the PRIMUS design: e.g. WILLIAM, waveform-based neural nets, new variants of neural-symbolic transformer architectures
- **Introduction of new theoretical and mathematical frameworks** for guiding the detailed implementation and harmonization of different AGI algorithms within the Hyperon/PRIMUS framework: the theory of quantale weakness as reviewed in the “Weakness is All You Need” manuscript, and the theory of transfer learning and cognitive synergy from the “TransWeave”

manuscript. These transition PRIMUS from a loose integration of complementary components into a tightly-integrated cross-AI-paradigm mathematical machine, configured for efficient execution on Hyperon infrastructure

We’ve been busy indeed ... and it’s been an extremely productive time to be working toward AGI!

1.6 From AGI to ASI

Hyperon and PRIMUS aren’t designed merely to achieve AGI—they’re engineered to safely evolve toward artificial superintelligence (ASI). This isn’t pure hubris; it’s a recognition that any successful AGI system must be able to improve itself, and those improvements must preserve human values and safety constraints.

The architecture supports this through several mechanisms:

1.6.1 Goal Stability Under Self-Modification

The system models goals as hierarchies of invariants—properties that must be preserved even as the system evolves. Using techniques from fixed-point theory and dynamical systems, we can prove that certain goal structures remain stable even under radical self-modification. This gives us mathematical confidence, not just hope, that an improving system will remain aligned.

1.6.2 Shared Algebraic and Dynamical Controls

Throughout Hyperon, the same mathematical structures govern different types of cognition. Weakness (simplicity) and geodesic effort aren’t just useful metrics—they’re universal currencies that ensure different cognitive modules compose safely. When the system modifies itself, these shared controls prevent any one component from consuming unlimited resources or diverging from established constraints.

1.6.3 Decentralized Infrastructure and Governance

Using capability-secured execution through RSpace/Rholang, the system supports multi-party evolution with full audit trails. Every modification can be traced, verified, and if necessary, reversed. This isn’t just about safety (though that’s indeed important) —it’s also about enabling collaborative development where multiple human teams and multiple AI systems can contribute improvements while maintaining system integrity.

2 Hyperon System Design

Modern commercial AI systems are often complex multi-component hybrids ... for instance, modern LLM-based chatbots, while they have transformer neural nets at the center, also rely on additional tools such as retrieval-augmented generation, knowledge graphs, external domain-specific search tools, formal-reasoning verifiers and so forth. However, existing hybrid AI systems all suffer from a fundamental architectural problem: they’re built as collections of separate components that struggle to communicate in a sufficiently semantically rich fashion. You might have a knowledge database over here, a neural network trainer over there, a reasoning engine in another corner, and an orchestration layer trying desperately to coordinate them all. But sharing only queries and results and crude context summaries does not allow the different components to synergize as well

as they need to – for that the different components would need to be able to share intermediate cognitive-processing states with greater richness and detail. This creates massive inefficiencies—data gets copied repeatedly, caches become inconsistent, and the interfaces between components become brittle semantic bottlenecks because different components are speaking different languages with only very crude inter-translation.

Hyperon takes a radically different approach. Instead of separate systems talking through APIs, everything—facts, rules, neural embeddings, activations, goals, even the schedulers themselves—exists as first-class objects in a single shared space called the Atomspace. This means all cognitive modules can directly see and manipulate the same structures, enabling forms of interaction that would be impossibly complex in traditional architectures.

2.1 The Technical Foundation

The Atomspace is implemented as a typed, content-addressed metagraph where every piece of information is an “Atom.” These Atoms can represent anything from simple facts to complex procedures, from neural network weights to control signals. The power comes from how they’re organized and accessed.

Behind the Atomspace sits a number of different back-ends with different properties; we will focus here heavily on MORK, a high-performance engine that makes the Atomspace approach practical. MORK provides a Merkle-DAG/PathMap trie structure that enables near-constant time lookups, lock-free updates for concurrent access, and memory layouts that support both symbolic operations and dense numeric computations. Future developments may augment MORK with alternative or superior Atomspace backends, but MORK is working now and has been found to display highly favorable performance characteristics for relevant algorithms and datasets.

The MeTTa programming language, a simple and elegantly abstract scheme for human and/or automated construction of programs in the form of coupled metagraph rewrite rules, encapsulates cognitive transformations on Atomspace. MeTTa programs are themselves parts of Atomspace, a deep self-referential recursion that enables both very concise and efficient implementation of complex cognitive algorithms, and system-level reflection that is critical for self-understanding, self-modification and the path to ethical superintelligence.

The system scales through DAS (Distributed Atomspace), which extends the same unified space across clusters using MongoDB/Redis backends. Meanwhile, RSpace/Rholang integration – via a “MettaCycle” infrastructure that compiles MeTTa into a combination of MORK and rholang operations – enables capability-secured execution for multi-tenant scenarios and blockchain-based provenance tracking.

2.2 The Space API: A Universal Interface

The Atomspace is Hyperon’s digital brain tissue—a uniform substrate where different types of processing and data can coexist, act and interact. But just as the brain has specialized regions, Hyperon has “Spaces”—specialized environments that can be plugged into the Atomspace while maintaining the same interface.

A Space might be an in-RAM knowledge graph adaptively updated via various reasoning and learning algorithms, a distributed database shard, a connection to a neural network service, or even a blockchain-based smart contract executor. The beauty is that MeTTa code (Hyperon’s native language) is in large measure Space-independent—pattern matching, rule firing, and data manipulation all work basically the same way regardless of the underlying implementation. Different spaces may also use different back-ends – some might use MORK, some might use say Jetta (now

in development) for running on Android devices, some might use a totally different floating-point-centric representation if they are focused on neural networks, etc.

2.2.1 How Spaces Work

The Space API defines a standard interface that any backend can implement. The core operations—match, bind, and rewrite—work consistently whether you’re accessing local RAM, querying a distributed database, or invoking a neural network.

Some example Space types that have proven interesting so far:

- MORK backed Atomspace for scalable in-RAM cognitive processing
- DAS for distributed storage and large-scale queries
- Rholang Atomspace for concurrent, capability-secured execution
- Neural Spaces that make deep neural networks appear as queryable Atomspaces

The Space abstraction layer means that cognitive processes can seamlessly span different computational substrates without special-case code for each backend.

2.3 MORK: The High-Performance Core

MORK is where much of the real work of Hyperon cognition happens. It stores the Atomspace metagraph in a highly optimized, content-addressed trie structure that provides both the performance needed for real-time cognition and the flexibility needed for diverse cognitive operations.

The key innovation is the PathMap/Merkle-DAG structure. Every Atom gets placed at a unique location in a prefix-compressed trie. Since nodes are immutable, updates create new versions rather than modifying existing ones. Writers prepare their changes as compact “deltas” that get merged atomically, which means readers always see consistent data even while multiple writers are active. The data structure is optimized to support efficient pattern matching and unification queries, which lie at the core of many AGI-oriented algorithms.

This architecture solves a critical problem: how to let dozens of different cognitive processes— inference, learning, pattern mining, attention—all operate simultaneously and efficiently on the same large-scale knowledge graph without creating synchronization bottlenecks.

2.3.1 Weighted Atom Sweeps: Cognitive Scheduling

Along with pattern matching and unification queries, MORK also supports alternate modes of access such as the Weighted Atom Sweeps (WAS) system. Think of it as combining garbage collection with attention management for a thinking system.

Each node in the PathMap carries an aggregate weight that bubbles up from its children. This creates a probability tree where you can efficiently sample Atoms according to any importance measure—recency, relevance, uncertainty, or any other metric your cognitive process cares about.

The system provides two main traversal strategies:

1. **Weighted random walks** that descend the tree by probabilistically selecting children based on their weights
2. **Chunked priority queues** that maintain sorted lists of high-priority regions for systematic processing

Since these operations are lock-free and updates are commutative, multiple sweep processes can run in parallel. This acts like an operating system scheduler for cognition-background inference can trickle through less-important regions while urgent pattern recognition focuses on hot spots, all without explicit coordination.

2.3.2 Executing Dense Kernels: ShardZipper and ByteFlow

MORK’s “standard” compute API handles sparse kernels such as pattern matching and unification queries very efficiently. Handling dense kernels will require additional work that is likely to be completed in 2026. We have articulated two complementary approaches to handling dense kernels:

ShardZipper is designed for deterministic batch processing. When you need to extract a large portion of the graph, run intensive computations on it (perhaps on a GPU), and then merge the results back, ShardZipper provides the machinery. It works by defining shards through hash prefixes, maintaining a “one-hole context” (technically, a McBride derivative) that lets you extract a shard, process it in isolation, and zip it back into the global structure with full Merkle integrity preservation.

ShardZipper can be implemented relatively straightforwardly against MORK as it exists now. ByteFlow takes a different approach, optimized for scenarios where hot regions drift over time; it would require more changes to underlying MORK infrastructure for efficient implementation, but once this would done, it would likely offer superior performance in many scenarios. The algorithm, continuously repacks frequently-accessed subtrees into contiguous parent-pointer blocks (PPTNs) that can be fed directly to kernels without pointer chasing. A routing table maps key prefixes to these packed blocks, and the system dynamically splits or merges blocks as access patterns change, all without requiring full re-serialization.

Roughly speaking, we might ultimately end up using ShardZipper when:

- Your computations naturally work on stable, well-defined regions
- You need deterministic, reproducible batch processing
- You want to distribute processing across multiple devices with clean boundaries

and using ByteFlow when:

- Access patterns change dynamically based on cognitive focus
- You need minimal latency for kernel operations
- You want automatic adaptation to changing workload patterns

However, we will understand the particulars of handling dense kernels better once we have proceeded with implementation and testing of this aspect.

2.4 Neural Network Integration

Hyperon’s approach to neural networks reflects a fundamental design choice: should neural computations happen outside the Atomspace (as service calls) or inside it (as native graph operations)?

Outside integration through Neural Spaces is the pragmatic choice for existing models. Large language models, vision networks, and other pre-trained systems run in their original frameworks but expose their embeddings and activations as Atoms. MeTTa code can query these neural representations, attach evidence and truth values to them, and use them in reasoning—all without modifying the underlying neural architecture.

Inside integration through QuantiMORK represents a more radical fusion. Neural tensors become multiresolution DAGs stored directly in MORK’s PathMap structure. Attention, convolution, and other operations run on these native structures, while MeTTa rules orchestrate when and where computation occurs.

2.4.1 The QuantiMORK Advantage

When neural networks live inside the Atomspace, remarkable things become possible:

- Pattern mining can identify important features that immediately become attention targets
- Symbolic proofs can guide where neural refinement is needed
- Computation can be selectively expanded only in uncertain regions
- Neural and symbolic processes share not just data but actual memory structures and caches

This deep integration can be achieved via “waveformer” approach, in which neural layers are structured via wavelet transforms, which have a hierarchical nature that naturally maps into MORK’s internal prefix-tree structure. Due to this structural correspondence, when we implement things this way, neural computations can flow through the same substrates as symbolic reasoning, enabling forms of neurosymbolic interaction impossible in traditional architectures.

2.5 Distributed Execution and Scaling

While it’s important to pack as much data and processing as possible into the RAM of a single machine (and MORK makes great headway in this direction), in the end real-world AGI systems will also need to scale beyond what a single machine can handle. DAS (Distributed Atomspace) extends Hyperon’s unified cognitive space across clusters while preserving the illusion of a single, coherent Atomspace.

DAS uses MongoDB and Redis to shard the metagraph, maintaining indexes that support efficient traversal, property queries, and pattern matching at scale. From the perspective of MeTTa code, there’s no difference between querying local Atoms and those stored across a cluster—the Space API handles routing transparently.

2.5.1 Decentralized Execution

For scenarios requiring multi-party collaboration or high-security execution, Hyperon has a “MetaCycle” layer that supports running MeTTa code as Rholang processes on RSpace. This provides:

- **Capability-based security:** Fine-grained control over what code can access
- **On-chain provenance:** Cryptographically verifiable execution history
- **Distributed consensus:** Multiple parties can contribute to cognitive processes

This isn’t just about scaling—it’s about enabling new models of collaborative AI development where different organizations can contribute cognitive modules while maintaining security and attribution.

2.6 State Management and Persistence

A thinking system needs to manage state carefully—not just for persistence, but for reproducibility, debugging, and collaborative development. Hyperon’s State Management Subsystem (SMS) provides a thin layer that treats engine internals as opaque while offering essential services.

The system captures state changes as `AtomSpaceUpdate` patches that bundle link insertions and weight modifications from each cognitive epoch. Four key artifacts get stored:

- **CheckpointRef**: Snapshot identifiers for full state recovery
- **UpdatePatch**: Incremental changes since the last checkpoint
- **ChangeDigest**: Summaries for quick difference detection
- **InfluenceSketch**: Approximate impact assessments for optimization

SMS uses CRDT G-counters for edge weights, ensuring clean merges across distributed replicas, while Rholang lens contracts provide capability-controlled access to specific query patterns and traversal depths.

2.7 Putting It All Together

Let’s trace through what happens during a typical neurosymbolic operation to see how these components work together:

1. **Selection**: A WAS (Weighted Atom Sweep) process identifies a hot region of the graph that needs attention, perhaps because new information has arrived or uncertainty is high.
2. **Layout**: ByteFlow’s routing table maps this region to a parent-pointer block containing the relevant Atoms in a kernel-friendly format.
3. **Computation**: A GPU kernel processes the block—perhaps running attention mechanisms or updating neural weights—and produces a compact patch describing what changed.
4. **Integration**: The host system replays this patch into the PPTN. If the changes exceed certain thresholds, ByteFlow might split or merge blocks to maintain optimal layouts.
5. **Propagation**: MeTTa rules detect the changes and trigger appropriate responses—perhaps PLN factor-graph messages need to propagate new evidence, or pattern miners should look for newly-emergent structures.

Throughout this process, everything stays in the same address space, uses the same indices, and maintains the same invariants. There’s no serialization boundary between neural and symbolic processing, no synchronization bottleneck between different cognitive modules.

3 The Language Stack

3.1 A Multi-Layer Approach to Cognitive Programming

Building cognitive systems requires balancing competing demands, on the developer-interfacing level as well as the underlying cognitive-dynamics level. Developers need a language that’s intuitive enough to express complex cognitive patterns easily. The system needs a compact, unambiguous

representation for compilation and distribution. The runtime needs low-level operations that can manipulate data structures efficiently. And increasingly, teams want to leverage Python’s ecosystem and LLM-assisted development.

Hyperon addresses these needs through a carefully designed stack of languages, each serving a specific purpose while maintaining semantic consistency:

- **MeTTa**: The high-level language where developers write abstract cognitive code
- **MeTTa-IL**: The intermediate representation used for compilation and optimization
- **MORKL/MM2**: Low-level languages that run directly on the graph structures
- **PyMeTTa**: A Python-compatible layer for rapid prototyping and integration

This multiplicity of linguistic tools isn’t about reinventing wheels—it’s about ensuring that code written at any level can be efficiently transformed and executed while preserving the unique requirements of cognitive computation.

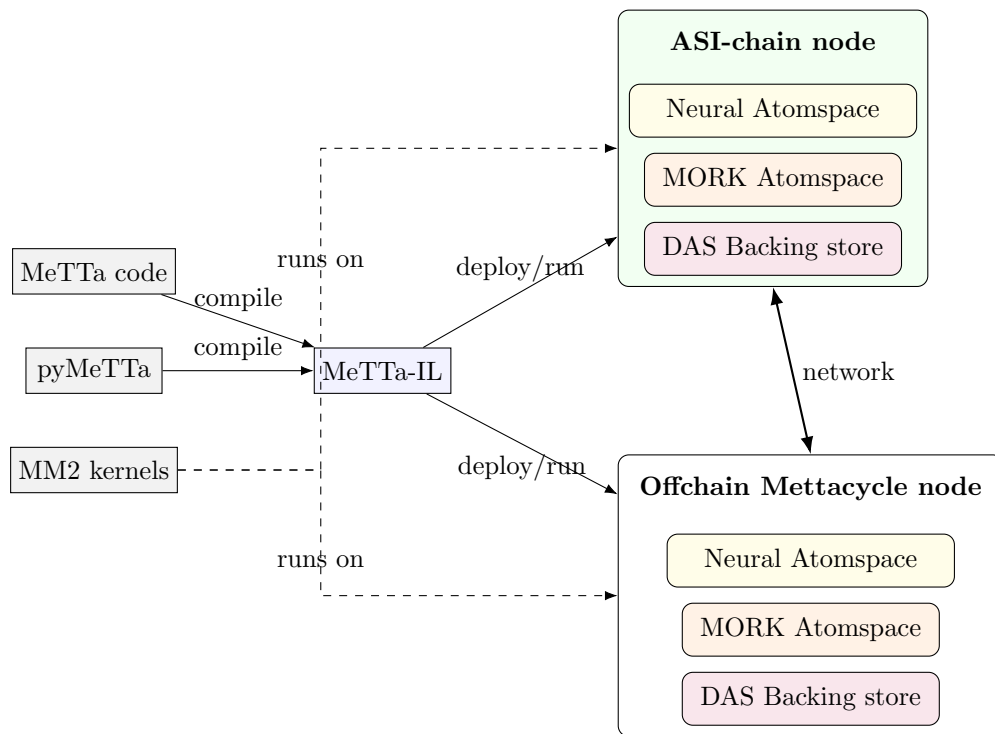


Figure 1: High-level software architecture: MeTTa/pyMeTTa compile to MeTTa-IL which leverages MORK Atomspace; MM2 runs on MORK Atomspace; both ASI-chain and Offchain Mettacycle nodes contain full Atomspace stacks with DAS backing. Nodes may in general contain numerous Atomspaces not just 3.

3.2 The Technical Stack

Each layer in the stack has precise responsibilities:

MeTTa provides a declarative, homoiconic pattern-rewrite language built specifically for Atom-space manipulation. It operates at a higher level of abstraction than other functional or logic programming languages, allowing flexible experimentation (by humans or AI modules) with a wide

variety of type systems, language constructs, logics and so forth – while still retaining speed and scalability in a wide variety of situations. It features committed-choice evaluation, explicit Space management, and built-in support for nondeterministic streams.

MeTTa-IL serves as the compiler-friendly intermediate representation derived from a Graph-Structured Lambda Theory (GSLT) description. This is what actually gets distributed, optimized, and compiled to various backends including MORK, Rholang, JAX, and Rust kernels.

MORKL/MM2 implements kernel-level typed operations that work directly with MORK’s data structures—factor-graph message passing, weighted sweeps, and ByteFlow operations for GPU/TPU acceleration. Highly performance sensitive algorithms that need to operate across MORK at scale may be most effectively implemented in MM2.

PyMeTTa and the metta-magic library (still under development, expected to launch in H2 2026) provide a restricted Python dialect that transpiles cleanly to MeTTa-IL, plus a comprehensive library suite. This enables notebook-based development, seamless LLM assistance, and integration with the broader Python ecosystem.

3.3 MeTTa: Cognitive Code as Graph Transformations

MeTTa takes a distinctive approach to programming: you write small equations, queries, and control patterns that match against subgraphs and rewrite them. The language keeps results as lazy nondeterministic streams until you explicitly collapse them—a design that naturally fits cognitive processes where multiple possibilities need to be explored in parallel.

When you write MeTTa code, you can target specific Spaces (local Atomspace, distributed DAS shards, or neural Spaces) simply by naming them. The same pattern-matching and rewrite rules work regardless of whether you’re operating on a toy knowledge base or a cluster-scale deployment.

3.3.1 The Execution Model

MeTTa-4 is a semantic model for MeTTa, intended to provide a clean, small-step operational semantics that aligns directly with the surface syntax. The core includes:

- **Committed choice by default:** Once a branch is selected, it proceeds deterministically
- **Delimited nondeterminism:** Explicit operators for forking and rejoining computation streams
- **Atomic Space operations:** The `for` construct provides transactional semantics across Spaces

The specification offers both an abstract machine view and an equivalent MeTTa-calculus formulation. The compiler pipeline cleanly separates kernel operations (rewrite/match/space ops), macro expansions (let/if constructs), and environment management (modules/tokens), enabling efficient compilation to the intermediate representation.

3.4 MeTTa-IL: The Compilation Bridge

MeTTa-IL (IL = Intermediate Language) exists because we need absolute clarity about what programs mean when they cross system boundaries. Every construct becomes explicit and typed in the IL, eliminating ambiguity about evaluation order, Space selection, or type conversions.

This intermediate form enables several critical capabilities:

- Generate both interpreters and type systems from the same specification
- Target multiple execution backends without semantic drift
- Apply optimizations with confidence that behavior won't change
- Verify safety and other properties before deployment

3.4.1 The GSLT Foundation

The system uses Graph-Structured Lambda Theory as its formal foundation. For any domain—whether it's normalization, satisfiability checking, or evolutionary computation—we can present a GSLT that captures its essential structure. From this GSLT, we mechanically derive both an interpreter that executes programs and a type system that validates them.

MeTTa-IL serves as the concrete syntax that these generators consume. The system actually maintains two GSLTs: a fine-grained one capturing implementation details and optimizations, and a coarse-grained one providing developer-facing types. A formal proof establishes the correspondence between them.

3.5 PeTTa and JeTTa

MeTTa-IL provides a robust and principled approach to MeTTa compilation, complete with a framework for proving correctness and safety in various senses – ideal for applications like smart contracts and AGI safety where clean provability is key. It also embodies highly robust concurrency in diverse scenarios via incorporation of the rho calculus at a low level. However, there may also be AI scenarios where these properties are not key and alternative ways of compiling MeTTa may be more convenient.

Jetta is a framework for compiling MeTTa to the JVM, leveraging the Kotlin infrastructure. This allows utilization of the JVM's highly sophisticated garbage collection framework, and also allows MeTTa code to run on Android devices.

PeTTa is a MeTTa compiler developed using SWI-Prolog (?PeTTa?) using recursion schemas, generally providing execution speed on par with modern Prolog and competitive with many functional-programming runtimes.

PeTTa can also connect with MORK on the back end which provides increased scalability. Careful evaluation is still underway, but anecdotally, at time of writing PeTTa can handle 50-100M Atoms in RAM on its own, on a powerful modern development machine ... but via PeTTa/MORK integration one can get a half billion Atoms in RAM.

PeTTa includes valuable though maximally sophisticated concurrency management, and does not attempt blockchain integration; however, MORK itself handles a lot of concurrent processing under the hood in the way it processes pattern matching and unification queries. Future experimentation will determine the use-cases where MeTTa-IL provides a practical performance advantage over PeTTa (a separate issue from the capability of MeTTa-IL to provide provable adherence to properties); one line of thinking is that MeTTa-IL will excel performance-wise for cases that involve a significant percentage of complex symbolic processing aside from MORK operations.

3.6 MORKL/MM2: Where Hot Loops Live

High-level languages are ideal for both human and AI programmers, in many senses. However, for effective AI system architecture, some operations ultimately need to run where the data lives, with minimal overhead and maximum parallelism. Factor-graph message passing for probabilistic

inference, ECAN attention sweeps, pattern mining iterations, and kernelized tensor operations—these are examples, and they form key aspects of the computational backbone of cognition in the PRIMUS framework.

While we could implement these in MeTTa, the performance would be inadequate for real-time cognitive processing. MORKL and MM2 provide the low-level primitives that achieve database-engine speeds while maintaining the semantic guarantees our logic requires.

MORKL is for efficient MORK database lookup – think SQL for MORK – whereas MM2 is for dynamic algorithms implemented against MORK at scale.

3.6.1 Core Operations

The MM2 layer implements multiple critical PRIMUS subsystems; as illustrative examples:

Factor-graph PLN encodes logical variables and rules as Atoms in the graph, with local potentials as typed functions mapping neighboring truth values through a product quantale structure. Messages traverse MORK using PathMap neighbor lookups, while the orchestration layer manages convergence, termination, and independence-error mitigation.

Weighted Atom Sweeps leverage the aggregate weight counters in each trie node to sample subgraphs proportionally to importance, age, or other metrics. The transform functions, written in MeTTa or PyMeTTa, apply local updates that propagate through zipper merges.

3.7 PyMeTTa: Developer Ergonomics

While many AGI researchers embrace high level and flexible languages such as LISP, Prolog or MeTTa, many AI application engineers prefer working in familiar imperative-language-based environments with robust tooling, visualization capabilities, and AI-assisted coding. PyMeTTa (in the early stages of development as time of writing) provides a Python-compatible surface that transpiles to MeTTa, enabling notebook-driven development without sacrificing the semantic guarantees of the core system.

The metta-magic library brings batteries-included functionality, e.g. covering :

- PLN inference operations
- Evolutionary algorithms (EDA, genetic programming)
- Pattern mining pipelines
- Algorithmic chemistry simulations
- MetaMo control interfaces
- DAS integration utilities
- *and more*

3.7.1 Clean Transpilation

PyMeTTa deliberately restricts Python to a subset that can be unambiguously converted to MeTTa-IL. This means explicit loop and conditional structures, typed decorators for function signatures, and clear module boundaries.

The benefit is substantial: code written in PyMeTTa compiles to the same MeTTa-IL that would be generated from native MeTTa, which can then run on MORK, Rholang, or any other backend

without requiring Python in the runtime. This preserves determinism and enables verification while providing Python’s development convenience.

3.7.2 Performance Integration

The metta-magic backend registry enables performance-critical routines to be compiled to native Rust or JAX/XLA kernels. The system JITs these during execution, crossing the Python-native boundary only once per cognitive step (typically 5-50ms), keeping orchestration overhead negligible.

The library provides comprehensive coverage across algorithms (genetic programming, EDA, PLN, mining, algorithmic chemistry), control systems (MetaMo, ECAN, schedulers), external integrations (DAS, avatars), and contract systems (Rholang compilation, on-chain proof checking).

3.7.3 LLM-Assisted Development

The system embraces AI-assisted coding as a first-class concern. PyMeTTa can be extremely robustly vibe coded as it’s python; modern LLMs can also directly code MeTTa or MM2 in many cases, and their skill here is improving rapidly.

3.8 Distribution and State Management: Handling Multi-Node Cognition

Real cognitive systems involve multiple machines, changing knowledge graphs, and teams of developers. The Hyperon State Management Subsystem provides essential services while treating engine internals as opaque, ensuring clean abstraction boundaries.

State changes are captured as AtomspaceUpdate macro-patches that bundle link insertions and weight modifications from each inference epoch. The system maintains four key artifacts per batch:

- **CheckpointRef**: Points to complete state snapshots
- **UpdatePatch**: Contains incremental changes since the checkpoint
- **ChangeDigest**: Provides quick difference summaries
- **InfluenceSketch**: Estimates impact for optimization decisions

Updates use CRDT G-counters for edge weights, ensuring clean merges across replicas without synchronization overhead. Rholang lens contracts provide capability-controlled access, restricting queries to specific templates and traversal depths.

For LLM and RAG workspaces, the same contract system handles vector and graph events. For core Hyperon operations, it bounds cold-replay time and prevents data exfiltration through crafted second-order patterns.

3.9 Sketch of the Development Workflow

An example development flow utilizing these tools might leverage the entire stack:

1. High-level logic gets written in MeTTa (or PyMeTTa for Python users), keeping streams lazy and collapsing only at measurement boundaries
2. Compilation produces MeTTa-IL, the distributable artifact that preserves semantics across backends

3. Hot loops call into MORKL/MM2 kernels that operate directly on ByteFlow blocks or ShardZipper shards
4. Background processing runs via Weighted Atom Sweeps, maintaining attention and performing long-range inference
5. Distribution can target a combination of MORK for performance and MeTTa-IL for decentralized, capability-secured execution

4 The PRIMUS Cognitive Architecture

Next we review the cognitive architecture that is the primary use-case of all this sophisticated “plumbing” – we give process-level overview of PRIMUS “as a mind,” followed by how the legacy and new components fit that flow and how Hyperon’s infrastructure makes it practical.

While summarized here from a pragmatic and algorithmic view, it is worth emphasizing that PRIMUS is a result of decades of deep thinking about cognitive-systems theory and philosophy of mind, cross-correlated with the performance strengths and weaknesses of modern compute hardware and modern computer science. The reader interested in the deeper roots of PRIMUS can look into the relevant literature from decades prior, such as *Toward a General Theory of General Intelligence* and *The Hidden Pattern*..

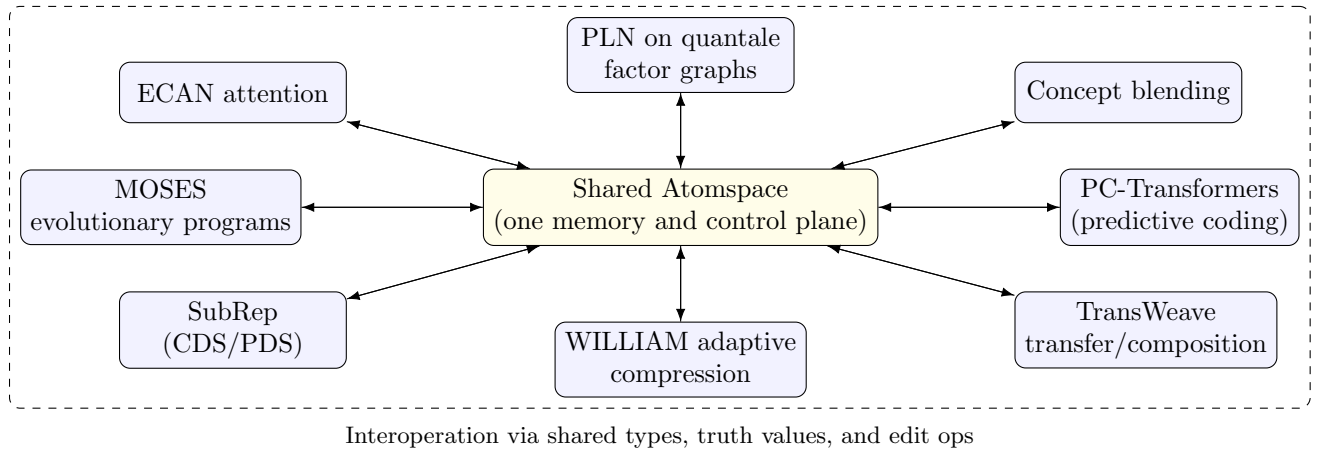


Figure 2: PRIMUS: legacy and new algorithms interoperate over a single Atomspace.

4.1 Two meta-dynamics: goal-directed and ambient cognition.

At any moment a PRIMUS instance is running two interleaved loops over a shared (generally: distributed and decentralized) Atomspace.

In the **goal-directed loop**, MetaMo maintains a small set of top-level motives (which it can revise on long timescales), then steers planning and decision toward actions that best advance those motives in the current context. This loop assembles and executes procedures by combining uncertain reasoning (PLN), learned skills (MOSES/GEO-EVO programs and neural controllers), and certified options (SubRep), checking progress and reallocating effort as evidence comes in.

In the **ambient background loop**, ECAN diffuses attention to high-value regions of memory and perception so the system can continually mine patterns, form and blend concepts, and run factor-graph PLN to tighten beliefs and discover new affordances—even when no immediate task

demands it. SubRep participates in both: it can certify options opportunistically in the background, or target option discovery during goal pursuit. This split–deliberate goal pressure plus continual exploratory thinking–captures PRIMUS’s day-to-day cognition.

4.2 How the pieces cooperate inside those loops.

In the goal-directed loop, MetaMo’s multi-objective motives frame “what counts as progress”; PLN supplies explainable chains that connect possible actions to expected outcomes; MOSES/GEO-EVO proposes compact programs/macros when the plan needs new skills; predictive-coding layers give fast forecasts and residuals; and SubRep gates any candidate option with simple certificates (CDS/PDS), keeping the skill library helpful and safe to compose. In the ambient loop, ECAN’s importance and urgency shape what gets computed; streaming **pattern mining** spots recurring structures that become new inference templates or skill hints; **concept blending** invents composite ideas and schemata; and factor-graph PLN consolidates what these discoveries imply. Across both loops, **TransWeave** provides the algebra so updates and transfers “almost commute”: $\text{learn?transfer} \neq \text{transfer?learn}$, with bounded order effects, which makes the goal and ambient processes reinforce each other instead of fighting.

4.3 Why this composes and scales on Hyperon.

Because Hyperon was designed largely with PRIMUS in mind, it is straightforward to implement PRIMUS in a fully Hyperonic way: everything—facts, rules, tensors, motives, certificates, transfer maps—lives as Atoms in one metagraph, manipulated by MeTTa rewrites. MORK turns that into speed: PathMap gives near-constant-time neighborhood access; Weighted Atom Sweeps schedule hot regions without global locks; and content addressing makes reuse and verification cheap. Distributed Atomspace (DAS) spans machines, and RSpace/Rholang enables capability-secured execution and audit. In practice, this lets PRIMUS run the ambient loop widely across a decentralized pool (pattern mining, background PLN, concept blending), while the goal loop binds a focused subnetwork to the active motives and pulls on ambient discoveries as needed.

4.4 Long-time Hyperon components in this picture

- **PLN** is PRIMUS’s uncertain reasoning backbone. In 2025 it runs as quantale-annotated **factor graphs**, so beliefs and their confidences move as local messages you can parallelize, pause/resume, and audit—usable both for deliberate proof-seeking in the goal loop and quiet consolidation in the ambient loop.
- **MOSES/GEO-EVO** discovers small, readable programs (controllers, macros) that slot directly into plans. GEO-EVO gives two-ended guidance (from current state and desired outcome), so evolution proceeds along minimum-effort paths, then exports skills back into Atom-space for reuse.
- **ECAN** allocates attention—short-term and long-term importance—over the metagraph, which PRIMUS uses as the scheduler for both loops. On MORK, these weights drive Weighted Atom Sweeps so high-value regions get first-class compute automatically. Potentially, incompressible-fluid-network based dynamics spread attention continuously behind the scenes, between ECAN steps, driving attention in a highly flexible and nuanced way.

4.5 Newly introduced ingredients that make PRIMUS even more of a coherent “mind.”

- **MetaMo** replaces single-reward heuristics with *motive geometries* (families of goals/constraints) and attaches simple certificates to choices, so trade-offs like performance vs. safety are explicit and auditable inside the goal loop—and informative to the ambient loop.
- **SubRep** turns subgoal/option learning into a planner-facing discipline: CDS (cone-dominant) and PDS (Pareto-dominant) admission rules, a co-learned Motive Decomposition Network, and composition guarantees. It works equally for neural controllers, logic macros, and evolved programs, and maps directly to MORK for scalable execution.
- **TransWeave** supplies the transfer/composition algebra so that cognitive steps across paradigms (neural, logical, evolutionary, program compression) “braid” with controlled order sensitivity and carry their certificates when you generalize to new tasks.
- **WILLIAM** adds a compression-driven pattern service that continuously proposes reusable motifs/programs and also acts as a system-wide efficiency oracle—pointing ECAN and schedulers toward regions with the highest compression/learning payoff.

4.6 Shared controls unify the two loops.

Two architecture-wide formal methodologies keep the goal and ambient processes aligned.

- **Geodesic control** (forward reachability backward usefulness) is the selection rule for inference, evolution, planning, and even self-modification;
- **Quantale-based weakness** is the Occam prior that favors simpler, more transferable structures across logic, neural, and program spaces.

Because both are encoded as weights/certificates on the same Atoms, they compose across modules and preserve guarantees during transfer—exactly what a practical, decentralized mind needs.

In short, PRIMUS “acts as a mind” by running a **goal-pressured loop** and an **ambient exploratory loop** over one memory and control plane, using old and new components as cooperating specialists, and relying on Hyperon’s substrate to make it fast, auditable, and distributable. The result is deliberation and discovery that constantly inform each other—guided by specific algebras (TransWeave) and metrics (weakness, geodesic effort) that keep the whole system stable as it learns and transfers what it knows.

5 Major Additions to PRIMUS Since 2023

We now review some of the core additions to the PRIMUS design, briefly mentioned above, that have been innovated since the previous Hyperon whitepaper from 2023. These are singled out here not because they are more important than the “good old stalwarts” of PRIMUS cognitive architecture such as PLN, MOSES and ECAN, but rather because they have not been covered in previous summary documentation.

The reader who wants a more comprehensive view of the cognitive processes behind PRIMUS is encouraged to dig into the *General Theory of General Intelligence* document from 2021 and follow the references therein. The AI algorithms and approaches discussed in this section fit very neatly into that framework, but are not mentioned there because they had not been invented yet in 2021.

5.1 Weakness Theory: The Mathematics of Simplicity

The idea that simpler explanations are better isn’t new—Occam’s Razor dates back centuries. What has been newly introduced to Hyperon/PRIMUS in early 2025 is having a single mathematical framework that makes “simplicity” precise and computable across all of cognition. Weakness theory provides this by measuring how much two items “fail to be distinguished” in a precise quantale-theoretic sense.

Michael Timothy Bennett’s formalization of set-theoretic weakness, in his PhD thesis, gave this direction a start (the idea here is, if one has multiple hypothesis explaining the same data, the “weaker” one that is true in more possible worlds will generalize better. There are then results showing that under appropriate assumptions, weaker hypotheses will have shorter description lengths in certain sorts of languages. “Quantale weakness” abstracts Bennett’s notion beyond sets to enriched quantales, a kind of formal category. The point of this that it lets us define “simplicity” in the native algebra of whatever algorithm or representation we’re using—logical proofs, probabilistic models, SVM margins, hypervectors, quantum operators, etc. Each of those worlds has its own way of combining and comparing things; a quantale captures that structure, and weakness says: a hypothesis is “simpler” when, under that structure, it rules out less or adds less extra structure. That yields an Occam-style heuristic that is not one-size-fits-all MDL, but a family of Occams—one per algorithmic context—each computable with the primitives the algorithm already uses. So instead of forcing every module to measure simplicity in bits, we can prefer weaker rules in logic via set/Heyting operations, prefer larger safe margins in SVMs, prefer lower-disturbance paths in Schrödinger-bridge dynamics, and so on—all instantiations of the same “prefer weakness” bias. The same weakness abstraction (but manifested in different quantales) that helps PLN choose inference steps also guides MOSES in program evolution, WILLIAM in pattern selection, and neural networks in avoiding overfitting.

Because these weaknesses live in different but compatible quantales, we can also compose them: map a logical weakness into a probabilistic one, relate margin-based weakness to information-theoretic weakness, or connect operator-level weakness to dynamical (bridge) weakness. Those functorial links let a planner, learner, or reasoner carry a single simplicity preference across neural, symbolic, evolutionary, and quantum submodules without flattening them into one metric. In practice, that means a cross-paradigm, holistic guidance signal: choose options, proofs, models, and policies that are jointly weak under the relevant algebras, and do so in a way that composes cleanly when modules hand results to each other. The result is an architecture-wide Occam’s razor that’s native to each algorithm yet aligned system-wide.

5.1.1 The PLN-Quantale Weakness Framework

In PLN logic, for example, we can assign each Atom a pair of valuations: one in a logic quantale capturing structural relationships, the other in a truth-value quantale capturing uncertainty. Weakness measures undistinguished pairs—cases where different items produce similar valuations.

As one example of the payoff of this mathematics for PRIMUS AGI, consider its use for implementing factor graph based control for PLN inference. Factor graphs have long been used for Bayes Net inference, but the standard approaches here don’t encompass full quantifier logic in a scalable way. Via constructing a quantale for quantified logic expressions, one can use factor graphs to propagate pairs (A, x) where A represents logical structure and x represents evidence. Combination uses the quantale product $\otimes$ at factors, marginalization uses the sum $\oplus$ at variables. The scheduler then prioritizes steps where weakness reduction per unit cost is largest—automatically balancing simplicity against completeness. Standard truth-value algebra integrates fully and ele-

gantly, with evidence counts, residuated implication, and data-processing inequalities all falling out naturally from the quantale structure.

As another example, if one adopts the Schrodinger Bridge quantale, one can then neatly envision a number of cognitive processes as following geodesic “shortest paths” through appropriate information spaces – where the “shortest path” means a path of “minimum representational effort” according to weakness theory in the given quantale.

5.1.2 Neural Network Applications

In neural networks, particularly when learning is coordinated via predictive coding rather than backpropagation, weakness often manifests as commutativity and monotonicity constraints. Different layer updates should commute (order shouldn’t matter much) and compose monotonically (adding layers shouldn’t cause wild behavioral changes). These properties can be expressed as residuation terms in the quantale, with penalties that stabilize training and reduce catastrophic forgetting.

This provides a unified view: the same mathematical principle that makes logical proofs prefer simple explanations also makes neural networks prefer smooth, generalizable solutions.

5.2 Geodesic Inference and Control

Traditional inference systems often waste effort oscillating between forward chaining from facts and backward chaining from goals. Geodesic control eliminates this inefficiency by maintaining both directions simultaneously and choosing steps that advance both.

The approach also enforces a critical safety property: conservation of evidence. Just as physical systems conserve energy, our inference system ensures that evidence neither spontaneously appears nor disappears—preventing both hallucination and information loss.

This approach has a general applicability, but is particularly elegant in its use within PLN (Probabilistic Logic Networks), a foundational part of the PRIMUS architecture that handles general deductive, inductive and abductive reasoning in an uncertainty-savvy way.

5.2.1 Mathematical Foundation

The system implements this through Schrödinger bridge geometry, defining an action functional that combines transport effort with prior regularization. The optimal solution has a beautiful property: at each point, the probability density is proportional to the product of forward and backward factors.

In practice, for PLN we approximate these factors through:

- Factor-graph messages (see below) for logical inference
- Monte Carlo estimates for meet probabilities
- Amortized neural predictors for complex domains

The control rule is straightforward: choose actions that maximize the change in $\log(f \times g)$ per unit cost, while maintaining approximately constant effort per step.

Evidence conservation emerges from a quantale-valued “generalized energy” that acts as a Noether invariant. We implement this using local evidence capsules (similar to CRDTs), overlap-safe merge operations, and content-addressed messages in the scheduler.

5.3 Fluid Dynamics for Attention: ECAN Meets Optimal Transport

A novel enhancement to ECAN (Economic Attention Networks) reframes attention allocation as incompressible fluid dynamics with optimal control semantics. Rather than treating attention as diffusing randomly through the knowledge graph, this approach models it as a conserved fluid whose flow is optimally steered toward goal-relevant regions.

The mathematical foundation connects the Hamilton-Jacobi-Bellman equation for optimal control on the group of volume-preserving diffeomorphisms to the Navier-Stokes equations governing fluid flow. Attention mass (STI values) becomes an incompressible fluid density $\rho(t, x)$ advected by a divergence-free velocity field $u(t, x)$, where u is derived from the optimal policy solving a boundary-value control problem.

5.3.1 Preserving ECAN Semantics with Enhanced Transport

This fluid dynamics layer doesn’t replace ECAN’s economic mechanisms but provides a sophisticated transport substrate. The system maintains all of ECAN’s invariants—conservation of attention budget, wage/rent/tax/welfare dynamics, and attentional focus selection—while adding goal-directed flow. The incompressibility constraint ($\nabla \cdot u = 0$) enforces strict budget conservation, while pressure acts as a shadow price for attention congestion.

Between ECAN’s discrete economic updates, attention evolves via the continuity equation with the velocity field computed from an HJB-derived optimal policy. This provides principled, goal-aware routing of attention that complements ECAN’s diffusion mechanisms. The approach is particularly powerful for credit assignment in multi-hop causal chains, where traditional diffusion may lag or blur the signal.

5.3.2 Implementation on MORK via ShardZipper

The dense numerical kernels required for advection-projection and HJB solution are efficiently implemented on MORK using the ShardZipper framework. This extracts locally-sweepable subtries, materializes them as contiguous structure-of-arrays buffers for GPU/CPU processing, runs the fluid dynamics kernels, and splices the results back into the metagraph. This enables scalable hybrid numeric-symbolic attention flow while maintaining the unified MORK storage substrate.

Importantly, TransWeave can be applied to transfer learned attention flow patterns between contexts. When the system discovers effective attention routing in one domain, TransWeave’s intertwining maps can adapt these flow fields to new but related problems, preserving the essential transport structure while adjusting for domain-specific features.

5.4 Incompressible-Fluid Networks

Continuing in the vein of optimal-transport-enhanced ECAN, the Incompressible-Fluid Networks approach treats neural network activations as a conserved fluid governed by optimal control principles, where information flows according to incompressible Navier-Stokes equations with velocity fields derived from solving HJB equations on the group of volume-preserving diffeomorphisms $\text{SDiff}(M)$. This provides principled, goal-directed routing of activations that maintains exact conservation (no magical creation/destruction of signal), with pressure acting as a Lagrange multiplier enforcing budget constraints and viscosity controlling exploration-exploitation tradeoffs.

5.4.1 Core Mathematical Framework

The foundation of In-Fluid-Net is a mapping from the Hamilton-Jacobi-Bellman (HJB) equation on the group of volume-preserving diffeomorphisms $SG = \text{SDiff}(M)$ to the incompressible Navier-Stokes equations on a manifold M . The value function $W(t, g)$ on SG satisfies:

$$\partial_t W + \nu \Delta_{SG} W - \frac{1}{2} \|\nabla_{SG} W\|^2 = V(g) \quad (1)$$

where V encodes task objectives and ν represents viscosity. The optimal control policy $U = -\nabla_{SG} W$ yields a velocity field that, after pushforward to physical space and Hodge-Leray projection to enforce divergence-free constraints, produces the incompressible flow field u satisfying:

$$\partial_t u + (u \cdot \nabla) u = -\nabla p + \nu \Delta u + f_V, \quad \nabla \cdot u = 0 \quad (2)$$

Neural activations are represented as a conserved density $\rho(t, x) \geq 0$ evolving by the continuity equation $\partial_t \rho + \nabla \cdot (\rho u) = 0$, ensuring exact conservation of total activation mass $\int_M \rho dx = \text{const.}$ The pressure p emerges naturally as a Lagrange multiplier enforcing incompressibility, acting as a shadow price for local congestion that automatically redirects flow to prevent bottlenecks.

Integration with ECAN The framework maps seamlessly onto Hyeron’s Economic Attention Networks (ECAN), preserving its core semantics while enhancing transport mechanisms. The crucial insight is that conservation of probability in ECAN (where STI/LTI values represent probabilistic importance measures that must sum to unity across the attentional focus) directly corresponds to conservation of mass in the fluid formulation. This allows the system to maintain ECAN’s economic accounting while replacing its isotropic diffusion with goal-directed, incompressible transport that routes credit along task-relevant pathways.

5.5 Integration with Predictive Coding

. Implementation leverages predictive coding for local learning, where each node maintains predictions of its activation based on neighbors, computing errors $e_i = a_i - \hat{a}_i$ and updating via local Hebbian-style rules without requiring global backpropagation chains. The system alternates between advection (moving mass via u), reaction (reducing prediction errors), and projection (enforcing incompressibility) phases.

5.6 Connections to Transweave and Neural-Symbolic Integration

The framework connects to the transweave perspective through its value-function structure on SG , enabling transfer learning across problem instances. Solutions can be “woven” between related tasks via HJB correspondences, with the incompressible pushdown providing reusable transport policies. The NS-level weave transfer conditions include geometry compatibility (volume-preserving isometries), forcing compatibility (velocity-independent sources mapping through cylinder potentials), and viscosity relations (time reparameterization or correction terms).

The In-Fluid-Diff extension adds generative modeling capabilities, combining incompressible transport with controlled diffusion: $dX_t = u_\theta(t, X_t, c)dt + \sqrt{2\kappa(t)}dW_t$, where the drift provides structured routing while diffusion enables density reshaping beyond volume-preserving rearrangement.

This approach achieves true neural-symbolic integration: symbolic ECAN rules govern economic accounting and structural updates, while continuous fluid dynamics handle efficient transport, with

conservation laws providing the mathematical bridge between discrete probabilistic importance values and continuous activation densities. The result is faster convergence, better credit assignment, and interpretable dynamics compared to pure diffusion-based or backpropagation-based approaches, while maintaining compatibility with Hypon’s existing attention allocation semantics.

5.7 Schrödinger Bridge Learning: From Abstract to Detailed Models

A fundamentally new learning paradigm has been introduced that leverages Schrödinger bridges—entropic optimal transport formulations that interpolate between probability distributions. Instead of the traditional approach of fighting overfitting through regularization, this method explicitly constructs distributions over both simple abstract models and complex high-fidelity models, then follows the entropic interpolation path between them.

This approach is particularly powerful for problems with natural hierarchical or multi-scale structure. The Schrödinger bridge automatically discovers a curriculum that progresses from coarse patterns to fine details, with the entropic regularization providing robustness while the optimal transport structure ensures efficiency.

5.7.1 Integration with Predictive Coding Networks

In predictive coding implementations, the bridge dynamics are approximated through local update rules at each layer. Each layer maintains parameter ensembles representing samples from the current distribution along the bridge. The system balances prediction error minimization with transport along the entropic interpolation, effectively implementing a time-varying complexity budget that naturally increases as learning progresses.

The mathematical elegance lies in how the same forward/backward factor decomposition that governs the bridge also aligns with predictive coding’s bidirectional message passing. This creates a unified framework where local computations collectively approximate the global entropic interpolation.

5.7.2 Evolutionary Programming Applications

For evolutionary programming (particularly in MOSES-style systems), the bridge framework replaces random mutations with “smart” mutations biased by the tilted kernel $K_t^*(x, y) = K(x, y)\psi_{t+1}(y)/(K\psi_{t+1})(x)$. This kernel reweights mutations based on their contribution to optimal transport toward the target distribution.

Early in evolution, mutations are conservative, preserving program structure while refining parameters. Later mutations make bolder structural changes as the bridge approaches the complex endpoint. The result is a principled curriculum from simple to complex programs, discovered automatically through the optimal transport computation rather than hand-designed schedules.

5.7.3 Conditions for Effectiveness

The bridge learning approach is most beneficial for problems exhibiting:

- Multi-scale separability, where coarse and fine structures can be meaningfully decomposed
- Compositional structure, where complex solutions build on simpler components
- Smooth interpolation meaningfulness, where intermediate models along the path are themselves useful

Under these conditions, the entropic interpolation provides advantages over standard regularization by explicitly modeling the trajectory through model space and using optimal transport principles to find efficient paths.

5.8 TransWeave: Intelligence Through Intertwining

TransWeave is a novel mathematical and conceptual framework addressing a fundamental challenge: how can a system reuse hard-won knowledge in new contexts? Traditional transfer learning often fails catastrophically—a solution that works perfectly in one domain becomes actively harmful in another. TransWeave provides mathematical guarantees about what will transfer successfully and what must be relearned.

The key insight is treating transfer as a composition problem. Rather than copying solutions wholesale, we learn mappings between task structures and compose them in ways that preserve essential properties. The same mathematical framework handles transferring policies, constraints, options, and even proof strategies.

5.8.1 Mathematical Foundations

TransWeave formalizes task morphisms as intertwining maps—structure-preserving transformations that approximately maintain key relationships. The critical advance is proving value suboptimality bounds: we can guarantee that transferred solutions won’t be worse than a specified threshold.

The framework also preserves weakness multiplicatively across chains of transfer. If each step in a transfer chain has a weakening factor, the composition has predictable total weakening. This lets us chain transfers through intermediate tasks while maintaining performance guarantees.

Admission certificates from SubRep propagate through the same machinery. If we know certain options are safe in one domain, TransWeave tells us which aspects of that safety transfer to new domains.

5.8.2 Impossibility Results and Selective Transfer

Not everything can transfer—and knowing what can’t is as important as knowing what can. TransWeave includes a hierarchical ICA (Independent Component Analysis) that identifies when solution components fundamentally cannot align between domains.

When the mathematics signals impossibility or near-singularity in the intertwining map, the system automatically falls back to selective transfer—moving what it can while marking specific components for relearning. This prevents the catastrophic negative transfer that plagues naive approaches.

5.8.3 Integration with Geodesic Control

TransWeave extends naturally to the geodesic inference framework used throughout PRIMUS. The product structure of forward/backward factors is preserved under transfer, as are evidence conservation properties. This means transferred solutions maintain the same efficient proof/planning characteristics as native solutions.

The MORK implementation can be used to store H-ICA components and learned mappings as Merkle-DAG objects with content identifiers. Factor-graph approximations of the forward/backward factors can then be used to enable efficient similarity searches and verification through Merkle proofs. The entire transfer process—analysis, matching, transformation, storage—operates on the same graph structures used for regular cognition.

5.8.4 Commutativity FTW

A more conceptual and formal way to look at TransWeave’s is to say that: Its big contribution is turning “how do we combine many learning/reasoning steps?” into an algebra with predictable behavior. It treats transfer as an intertwining between tasks—roughly, “do the update, then map” $\approx$ “map, then update”—so many operations **almost commute** with small, bounded order effects.

That lets you reorder pipelines like learn $\rightarrow$ transfer or transfer $\rightarrow$ learn without derailing results, and even run modules asynchronously while keeping them consistent. In practice this spans predictive coding updates, logical inference, evolutionary search, compression, and subgoal options: each admits a geodesic/bridge form, and the corresponding transfers compose with controlled commutator gaps rather than ad-hoc hacks. The payoff for Hyperon is a principled way to schedule and parallelize cognition across Spaces and PRIMUS modules while preserving value structure and performance guarantees.

An elegant aspect of all this from a mathematical point of view is that all these “almost commutative” pieces don’t just chain—they **braid**. TransWeave’s operator calculus shows how composite transfers obey braid-like laws (a Yang-Baxter style picture, formally speaking), so multi-step pipelines can be re-ordered along different strands with predictable, bounded change. Weakness (our quantale-based simplicity bias) threads through the braid: its loss multiplies across a chain, giving a clean book-keeping of “how much generality we’ve spent” as we interleave modules. The result is a *holistic* composition story: transports that minimize commutators are more freely re-orderable; weakness certificates survive composition; and when the math flags non-commuting or impossible segments, the system falls back to selective transfer rather than forcing brittle reuse.

For Hyperon specifically, this means PRIMUS components now plug into a **single operator layer**: predictive-coding learners, PLN inference, MOSES/GEO-EVO, WILLIAM/AdaptiMORK compression, and SubRep options can be woven in any pragmatic order with bounds on order sensitivity. SubRep’s admission certificates and multi-goal motives (MetaMo) ride the same machinery, so “what’s safe/useful here” carries forward when you jump tasks or swap algorithms; geodesic inference/control properties are preserved under transfer; and MORK stores the pieces so verification is merkle-provable. In short: TransWeave adds a governed algebra of composition—chains that commute up to measured error, braids that keep pipelines stable—which upgrades Hyperon from a powerful parts kit to a *coherent* system for scalable, modular, cross-paradigm cognition.

5.9 SubRep: Integrating Subgoal Decomposition and Representation Learning

SubRep is a new way of turning “find the right subgoals” into a disciplined, auditable process that works across neural, logical, and evolutionary modules. Instead of guessing which skills to learn, the planner proposes candidate options (controllers, macros, programs) and SubRep admits only those that provably help on the current slice of motives. Two simple tests do the screening: Cone-Dominant Subtasks (CDS) keep options that improve the planner’s backed-up value for *all* weights inside a learned motive cone; Pareto-Dominant Subtasks (PDS) keep options that improve at least some representative weights without unacceptably hurting the others. A small Motive Decomposition Network (MDN) co-learns that motive slice from the same residuals used to train option models, so the “what we care about” geometry tightens as the system gains experience. Every admitted option comes with a certificate (margin or Pareto gap) that survives composition inside the planner, so reuse is safe by construction.

Practically, this plugs straight into Hyperon/PRIMUS: predictive-coding heads supply the “what happens next” summaries (expected immediate payoff and expected successor features), logical macros and evolutionary programs export the same summaries, and the SubRep gate treats

them uniformly. Because the tests are local and cheap (closed-form for box-cones; small covers for Pareto), we can run them online and keep the option library compact. The result is a cross-paradigm option pipeline that grows only when a candidate demonstrably helps—and it keeps helping after we compose it with others.

Compared with “Hyperon 2023,” this is a step-change in how we manage goals and skills. The 2023 design outlined reinforcement learning alongside broader goal and attention processes, but it did not yet provide a unified, certificate-driven method for learning and vetting subgoals across modalities. SubRep adds that missing operator layer: formal admission rules (CDS/PDS), a co-learned motive geometry (MDN), and planner-safe composition guarantees, all wired into the same Atomspace/MORK substrates. In the 2025 stack we present MetaMo (motive management) and SubRep together as first-class regulators, so motivation, subgoals, and learning share one algebra and one audit trail.

Relative to standard RL, SubRep goes beyond single-reward shaping, option-critic variants, or “reward-respecting subtasks.” Instead of tuning a subtask to one scalar reward (and hoping it transfers), we admit options that are robust across a *family* of motives, or explicitly Pareto-good on a small cover when trade-offs matter. That shift—plus the use of learned motive cones/covers—prevents brittle skills, reduces negative transfer, and lets neural, logical, and evolutionary generators be screened and composed by the same planner. In short: SubRep makes subgoal discovery multi-objective, certifiable, compositional, and neurosymbolically native.

Finally, because SubRep’s tests are expressed in the same language the rest of Hyperon speaks (backed-up values over Atomspace features), they benefit from—and contribute to—the broader neurosymbolic loop: pattern mining finds reusable structures, predictive coding supplies fast updates, and weakness-based regularization prefers simpler, more general options. Put together, this is how Hyperon 2025 moves from “we can learn skills” to “we can justify, reuse, and scale them safely.”

5.10 MetaMo: A Compositional Motivation Architecture

Most contemporary AI systems treat motivation as a scalar reward signal—a single number that drives behavior. MetaMo, a newly designed general-purpose motivational framework for Hyperon, recognizes that real motivation involves complex interactions between appraisal (understanding the situation), decision-making (choosing actions), and goal management (maintaining coherent purposes over time), all carried out in the interest of pursuing a shifting collection of multiple motives.

(MetaMo is a natural complement to the SubRep formal framework for subgoals and options – which as we have seen provides mathematical certificates that guarantee when a subgoal genuinely serves a larger purpose (such as one of MetaMo’s motives) and when component solutions can be safely composed.)

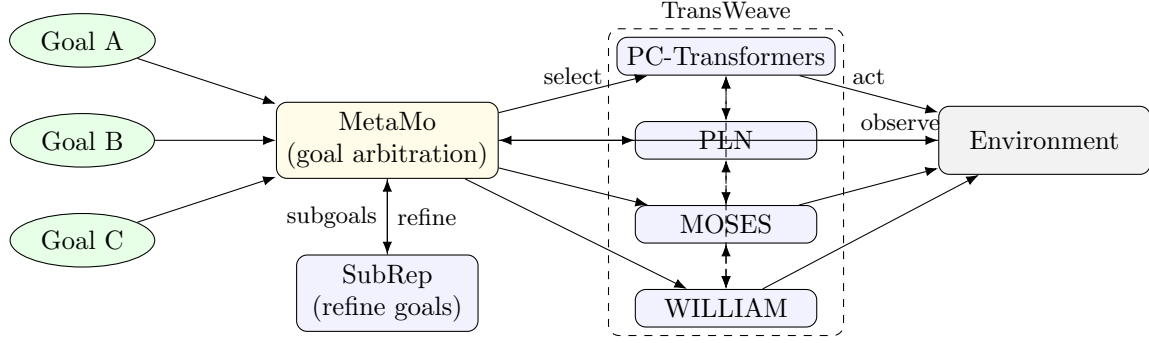


Figure 3: Goal orientation: MetaMo arbitrates among goals, consults SubRep for refinement, coordinates multiple methods for choosing and learning procedures to achieve goals, and leverages TransWeave for cross-method transfer.

5.10.1 The Mathematics of Motivation

MetaMo models cognitive motivation through a pseudo-bimonad structure—a mathematical framework that sounds esoteric but captures something profound about how appraisal and decision-making interact.

Appraisal is modeled as a comonad Ψ that extracts context and meaning from situations. Decision-making is a monad D that constructs action sequences. These are linked by a lax distributive law that captures how “feeling then choosing” differs slightly from “choosing then feeling”—a asymmetry that explains many quirks of motivated behavior.

The appraisal and decision processes are carried out in the interest of achieving a fuzzy set of top-level motives, which in general may itself evolve via the systems self-referential activity.

The composite structure $F = D \circ \Psi$ acts as a pseudo-bimonad, enabling nested cycles of appraisal and decision while maintaining mathematical coherence. The framework includes contractive update laws that keep trajectories within safe regions and uses topological arguments to guarantee that target states remain incrementally reachable.

5.10.2 Operational Design

MetaMo embodies five key principles that elevate it beyond traditional reward-based systems:

1. **Modular appraisal and decision** - These remain separate but coordinated
2. **Homeostatic stability** - The system resists runaway dynamics
3. **Reciprocal state simulation** - It models how its actions affect others
4. **Motivational compositionality** - Goals can be decomposed and recombined
5. **Incremental embodiment** - Abstract values connect to concrete actions

In Hyperon, MetaMo controls exploration budgets, manages even-cost geodesics for planning, and interfaces with SubRep to validate when options genuinely serve their declared purposes. The result is a motivation system that can pursue complex hierarchical goals while maintaining stability and alignment.

5.10.3 A Step Beyond OpenPsi and other Earlier Motivational Frameworks

MetaMo is in some ways a direct descendant of the OpenPsi motivational system we experimented with in OpenCog. OpenPsi gave PRIMUS/OpenCog a practical “drives and urges” core: homeostatic needs, curiosity, social reward, urgency/importance, and attention dynamics that select goals and allocate compute. That schema proved its worth in early neurosymbolic agents, but it was mostly hand-tuned, essentially single-objective at decision time, and hard to audit when many values and constraints collided. As systems grew to span predictive coding, logical inference, evolutionary program search, and compression, those ad-hoc priorities became a bottleneck.

MetaMo turns motivation into an explicit, learnable, and auditable object. It represents what the system cares about as *motive geometries* (families of goals and constraints rather than a single scalar), learns their structure from experience, and attaches simple, checkable “certificates” to choices the agent makes. That lets MetaMo arbitrate real trade-offs—performance vs. safety, speed vs. generality—without collapsing everything to one number. It’s built to compose, so when it plugs into SubRep (for certified subgoal/options) and TransWeave (for order-robust learning/transfer), the same motive signal guides neural, symbolic, and evolutionary modules consistently. In practice, you still get the familiar OpenPsi behaviors—arousal, urgency, curiosity—but now they sit inside a multi-objective, cross-paradigm framework with clear knobs, logs, and guarantees. In short: MetaMo doesn’t replace OpenPsi so much as *complete* it, evolving Hyperon’s motivational layer from useful heuristics into a systemwide, verifiable guide for cognition.

5.11 Algorithmic Chemistry: Computation via Reaction Networks

Imagine a computational soup where rewrite rules behave like molecules—combining, reacting, and evolving based on how well they predict and achieve goals. That’s algorithmic chemistry: a framework where computation emerges from the interaction of simple rules rather than being imposed by fixed algorithms.

ActPC-Chem (Active Predictive Coding Chemistry) adds goal-directed dynamics to this soup. Rules that successfully reduce prediction error or advance instrumental goals become more active, while unsuccessful patterns decay. The result is a self-organizing system that discovers effective computational strategies through evolution rather than design.

5.11.1 Technical Architecture

The system treats both data and models as evolving metagraph-rewrite patterns. Each rule carries a weight updated through discrete active predictive coding, balancing epistemic value (reducing uncertainty) with instrumental value (achieving goals).

Updates use discrete natural gradients based on optimal transport geometry—a mathematical framework that ensures stable learning even in this highly dynamic environment. Rather than traditional backpropagation, changes flow through the rule network following geodesic paths that preserve essential structure while enabling rapid adaptation.

5.11.2 Integration with PRIMUS

ActPC-Chem doesn’t replace other PRIMUS components—it provides a creative substrate they can build upon:

- AIRIS uses ActPC-Chem’s discovered structures as raw material for causal models
- PLN adds logical constraints that guide which rules can combine

- The system can implement transformer-like architectures where traditional backprop is replaced with rule-guided ActPC updates

A worked example traces a “virtual robot bug” learning to navigate, showing how rules emerge, combine, and stabilize into effective behaviors. The same framework scales to language modeling, where ActPC dynamics replace traditional gradient descent while maintaining compatibility with AIRIS/PLN for structured reasoning.

5.11.3 Why Hyperon?

This approach only works because Hyperon provides a unified substrate. Rules, tensors, and truth values all live in the same Atomspace/MORK structure, sharing IDs, caches, and schedulers. The Weighted Atom Sweeps system naturally handles the stochastic dynamics of the chemical soup, while weakness theory provides selection pressure toward simpler, more general solutions.

5.12 WILLIAM-on-MORK: Efficient Adaptive Compression for General Cognition

WILLIAM is a novel adaptive-compression-based AGI approach introduced by Arthur Franz, which is now being integrated into Hyperon in a fashion enabling very efficient execution on the MORK infrastructure.

WILLIAM embodies a fundamental principle: the patterns worth remembering are those that compress our experience most effectively. Think of it as a cognitive feature detector that continuously asks “what’s the simplest explanation that captures what I’m seeing?” By moving WILLIAM directly into MORK’s infrastructure, we’ve transformed this from a batch analysis tool into a real-time guide for both symbolic reasoning and neural processing.

The impact extends beyond pattern discovery. When WILLIAM identifies high-value structures, they immediately become available to guide proof search, focus neural attention, and prioritize which computations deserve scarce resources. The system achieves this without hard-coded heuristics—compression naturally identifies what matters.

5.12.1 Technical Implementation

MORK’s trie nodes now carry additional instrumentation: local occurrence counts, subtree totals, compression-gain sums, and ranked lists of top-k children by various metrics. This seemingly simple addition enables WILLIAM to expose weighted iterators that return heavy subpatterns directly from any point in the graph—no global scans required.

The API surface remains minimal and intuitive, e.g.:

- `iter_prefix_topk(prefix, k)` returns the k most important patterns under a prefix
- `iter_any_topk(k)` finds globally significant patterns
- A validation API records actual compression gains when patterns are applied

These iterators serve multiple masters. PLN uses them to prioritize inference on high-value subgraphs. Backward chaining follows “heavy edges” that are likely to succeed. Schedulers allocate resources based on compression-adjusted priorities. The implementation handles concurrency through per-core write buffers for counts, with wait-free readers using snapshot/RCU techniques.

5.12.2 Neural Network Acceleration

WILLIAM displays a strong potential for elegant integration with transformers and predictive coding networks. Applied to the internals of a neural system, especially one doing local learning (with their greater propensity toward compositional representations), it has the potential to find "heavy-hitter features" and uses them to:

- Prioritize which attention heads and tokens receive computation
- Trigger on-demand refinement only where uncertainty justifies the cost
- Prune low-value frequency bands while preserving model capability

The weakness prior provides the theoretical foundation, explaining why simpler patterns generalize better. Rather than ad-hoc sparsification rules, we have a principled framework that applies uniformly across symbolic and neural domains.

5.13 How These Advances Work Together

These recent cognitive-design additions we've reviewed aren't independent improvements—they form a coherent enhancement to PRIMUS's capabilities:

1. **Weakness Based Control** selects regions needing attention based on importance and uncertainty
2. **WILLIAM** identifies compression-worthy patterns in those regions
3. **Fluid-Dynamic ECAN and Learning** route attention optimally to where it's causally useful
4. **PLN** and **ActPC-Chem** refine understanding using both logical and chemical dynamics
5. **Schrödinger Bridge Learning** provides principled curricula from abstract to detailed models
6. **MetaMo/SubRep** evaluates which discovered options serve current goals
7. **TransWeave** determines what can be reused when facing new challenges
8. **PC layers** scheduled by WILLIAM and regularized by weakness theory process neural aspects

Throughout this flow, the same mathematical principles apply:

- **Weakness** governs all simplicity decisions: PLN scheduling priorities, WILLIAM's pattern selection, TransWeave's transfer filters, and PC network regularization all use the same quantale-based framework.
- **Geodesic control** guides all planning: the $f \cdot g$ product structure appears in inference, evolutionary search, transfer learning, Schrödinger bridge interpolation, and even motivational decisions.
- **Optimal transport** unifies dynamics: fluid-dynamic attention routing, Schrödinger bridge learning trajectories, and ActPC-Chem's gradient flows all leverage entropic transport principles.

- **TransWeave based knowledge transfer** spans cognitive components by viewing diverse cognitive algorithms effectively as approximate stochastic dynamic programming and guiding their interoperation accordingly
- **Content addressing** unifies storage: all components—templates, certificates, chemical rules, transfer maps, bridge potentials—are CID-addressed MORK objects, making reuse and verification constant-time operations.

The integration of these advances in a common mathematical and conceptual framework enables capabilities that none could achieve alone:

- Patterns discovered by WILLIAM become chemical rules in ActPC-Chem
- Chemical dynamics discover structures that TransWeave can transfer to new domains
- Transferred solutions maintain SubRep certificates, preserving safety guarantees
- Attention flow patterns learned through fluid dynamics can be transferred via TransWeave to new contexts
- Schrödinger bridges provide curricula that guide both neural and evolutionary learning
- Weakness regularization ensures that increased capability doesn’t mean increased brittleness
- MetaMo’s motivation system coordinates all these processes toward coherent goals

5.14 The Path Forward

These new additions transform PRIMUS from a capable cognitive architecture supporting flexible cross-paradigm integration into something more ambitious: a system with a specific methodology and mandate for integrating the semantics and dynamics of different cognitive algorithms and the transfer of all sorts of knowledge between algorithms and tasks.

The remarkable aspect here isn’t any single capability—it’s that they arise from shared mathematical foundations rather than ad-hoc engineering. This suggests we’re on the right track: not just building intelligence, but discovering the principles from which intelligence naturally emerges.

6 Re-Visioning of Longstanding PRIMUS Components

While the new additions to the PRIMUS framework are exciting, much of the cognitive heavy lifting is still done by stalwart components of the PRIMUS design which have been with us since OpenCog Classic (and in some cases the Novamente Cognition Engine and Webmind AI Engine before that). However, these venerable AI methods have acquired significant new aspects in the 2025 version of Hyperon/PRIMUS, both on the high-level control level and the low-level implementation level. They are being integrated into the geodesic control framework, for use with the compositional weakness, TransWeave and MetaMo approaches. And they are also being implemented in MORK-customized ways for scalable in-RAM efficiency (a process known informally as “Morkification”).

6.1 PLN (Probabilistic Logic Networks).

PLN is PRIMUS’s uncertain reasoning layer: it treats knowledge as a typed metagraph of propositions and rules, and carries both *what* we believe and *how strongly* we believe it via simple, interpretable truth values (strength and confidence). Alongside its sophisticated uncertainty management, PLN’s design has always emphasized two requirements—flexible “omni-ward” chaining (forward, backward, inward, outward) and programmable inference control—so the system can steer effort where it matters, not just produce long proofs. The current PLN approach keeps that spirit but streamlines the implementation by adding new options like guiding chaining geodetically, or implementing inference as message-passing on a factor-graph.

In the new PLN we can implement inference as “geodesic control”: maintain both forward factors (from premises) and backward factors (from goals) and move along steps that progress both—an instance of the Schrödinger-bridge view where optimal paths conserve evidence and bound order-effects. This makes PLN’s updates naturally interleave with other learners (e.g., neural critics, evolutionary search) under shared intertwining laws.

To implement PLN natively on MORK, one can either use backward or forward chaining processes that traverse the MORK prefix trees at a low level (incorporating geodesic control heuristics at this level as well), or one can lay out a factor graph atop MORK’s prefix trees and leverage this for a more decentralized and bottom-up form of control.

6.1.1 PLN Control via Quantale-Annotated Factor Graphs

“Classical” PLN tended to chain inference rules sequentially— which is very appropriate for many cases (e.g. when one has a particular focused question one badly wants to answer – but even with geodesic guidance this can be inefficient for large-scale intuition-building “background reasoning” on large knowledge bases. Factor graphs offer a better approach for this case: they let us propagate truth values and evidence as local messages through the graph structure, enabling parallel processing and natural suspension/resumption of inference.

One key insight is keeping logical structure and uncertainty measures together through the message propagation process. Rather than maintaining separate systems for “what we know” and “how certain we are,” every piece of information carries both aspects through the same computational flow.

The Technical Implementation We encode truth values (strength and confidence) as elements in a commutative quantale—a mathematical structure that elegantly handles both logical operations and uncertainty propagation. By pairing this with a logic-formula quantale, we create a combined product quantale that travels through the factor graph carrying both structural and probabilistic information.

In practice, each proposition becomes a `VariableAtom` carrying truth value payloads, while each inference rule instantiation becomes a `FactorAtom` with a local potential function that maps input truth values to outputs. `FactorVarLinks` connect these elements, and MORK’s `PathMap` structure makes neighbor lookups nearly instantaneous.

The orchestration layer handles the messy realities of distributed inference: damping to prevent oscillation, termination detection, independence-error heuristics to avoid double-counting evidence, and capsule logging for debugging and provenance.

6.2 Evolutionary Program Search

MOSES is Hyperon/PRIMUS’s automatic program learning methodology, bringing estimation of distribution algorithms together with evolutionary programming. It searches over spaces of small, human-readable programs—controllers, predicates, macros—that live directly in the Atomspace. It organizes the search into local “demes” (nearby template families), learns which parameters co-vary, and samples promising variants; the GEO-EVO upgrade adds two-ended guidance: search forward from what we have and backward from what we want, meeting along minimum-effort paths rather than wandering randomly. Practically, that means faster discovery of reusable cognitive skills that plug back into reasoning and planning without being opaque.

MOSES embeds naturally into MORK as its natural use in a Hyperon context is to evolve programs that are themselves represented at MORK subtrees. Program templates, factor-graph priors, and deme statistics are content-addressed Atoms. MORK deduplicates near-identical candidates, indexes parameter neighborhoods for fast sampling, and streams only the “delta” of changed subgraph, keeping throughput high.

TransWeave enriches the picture by casting evolution as stochastic dynamic programming and supplying typed transfer operators between logic, evolution, and neural value learners (“do update, then map” ? “map, then update”). PLN subgoals can bias evolutionary variation; evolutionary regularities feed back as new rules; neural critics provide amortized value estimates—and the commutator gaps are bounded so you can reorder steps without breaking progress.

6.2.1 GEO-EVO: Evolutionary Program Search

Deep integration of MOSES with TransWeave yields GEO-EVO – an enhancement that adds bidirectional guidance to the evolutionary picture: forward search from what we have and backward “teleological” search from what we want, meeting in the middle along minimum-effort paths. This isn’t just random mutation slowly veering toward acceptable solutions –it’s principled exploration that respects both the structure of program space and the goals driving the search. This rests on an elegant theoretical foundation of viewing evolution as an optimal transport problem—finding minimal-effort paths through program space that connect current capabilities to desired outcomes.

GEO-EVO’s two-ended guidance maintains forward reachability factors (can we get there from here?) and backward compatibility factors (is it useful for our goals?). The system expands search where the product of these factors increases most per unit effort, while maintaining roughly constant computational cost per step. The mathematics guarantees that intermediate steps balance exploratory diversity with goal-directed progress.

6.3 ECAN (Economic Attention Network).

ECAN is Hyperon’s primary, generic attention allocator. Other cognitive components may have their own internal attention allocation but there is still a need for higher-level attention allocation that helps guide attention across all subprocesses in the system. ECAN does this by spreading two “currencies”—short-term importance (STI) and long-term importance (LTI)—over the metagraph, forming an attentional focus that prioritizes computation and memory movement. The design is intentionally simple: importance rents, diffusion along links, and accumulation of durable relevance, tuned so the system neither fixates nor thrashes. That mechanism proved powerful in earlier OpenCog work and carried into Hyperon with better scaling and clearer knobs.

ECAN’s importance-based selection can be implemented with high efficiency on MORK via embedding statistical records related to importance sampling directly on MORK nodes. And information-geometric (geodesic) methods have been used to accelerate ECAN since the OpenCog

Classic days, which fits right into the TransWeave perspective. Fluid-dynamical methods may also be used to guide attentional flow on a more microscopic time-scale inbetween ECAN iterations.

6.4 Pattern mining on general graphs.

Pattern mining is an indispensable low-level cognitive heuristic that finds reusable structures—templates that recur across contexts—and uses them to prime reasoning, suggest subgoals, and compress knowledge. The approach we have taken to this in Hyperon is incremental and stream-based: generate abstract templates, specialize, filter by support, expand conjunctions, and rank by “I-surprisingness”—but keep everything as nondeterministic streams until the final tally. That makes it natural to run continuously as knowledge drifts, and to reuse the results directly inside PLN and other cognitive methods.

To extend Pattern Mining into TransWeave, we can view mining as two-ended geodesic discovery between a data-driven prior over candidate patterns and a goal-driven posterior over useful patterns for downstream tasks. Under this lens, mining steps commute cleanly with BD/bridge operators and can be interleaved with reasoning or evolution without losing alignment.

Efficient MORK implementation of pattern mining applies especially in the case where the patterns searched are naturally expressed as subtrees. In this case, matches are PathMap traversals; candidate templates are stored once and referenced many times; and counts/metrics are kept as capsule summaries at the nodes. Weakness-guides search lets miners “hover” over hot subgraphs and spread out opportunistically when attention shifts, so throughput scales with graph size without global scans.

6.5 Concept blending.

Concept blending is a primary heuristic for creativity – it creates novel ideas by merging properties from two or more source concepts so that the result shows useful emergent structure (think “sky-garden” = aerial platform + botanical garden). In Hyperon this is not a bag-of-words trick: blends are metagraph constructs that can be tested by the same evaluators that rate patterns, proofs, or programs, which makes blending a first-class, auditable operation rather than a one-off heuristic.

In a TransWeave setting, blending becomes a transport path in concept space: choose maps that minimize effort while preserving the constraints each source contributes. Because TransWeave supplies braid/near-commutation rules, blending can be interleaved with pattern mining, inference, or program search and still remain consistent with the system’s value geometry.

To make this work efficiently on MORK, sources, alignments, and resulting blends are content-addressed; shared sub-parts deduplicate automatically. Transport metadata (who contributed what, with what cost) is stored alongside the blend so future transfers can reuse the same alignments quickly.

7 Neural-Symbolic Synergy in Hyperon

While LLMs and other standard modern neural net architectures appear inadequate for human-level AGI, due to their lack of internal abstraction leading to an inability to make creative leaps too far beyond training data, there is no doubting their impressive practical capabilities in many regards. These capabilities could surely be replicated via non-neural architectures as well; however, there is a strong pragmatic motivation to keep rolling with what works and integrate it with other things that work in other ways. This militates toward hybrid AGI architectures, e.g. neural-symbolic, neural-symbolic-evolutionary, and so forth.

Crudely, we may say current neural networks excel at pattern recognition, handling ambiguity, and learning from examples. Symbolic systems currently shine at explicit reasoning, structured manipulation, and explanatory transparency. Running them in isolation means constantly shuttling data back and forth, losing context in translation, and missing opportunities for mutual enhancement.

Hyperon eliminates these barriers by making neural and symbolic components first-class citizens in the same computational space. Features, rules, proofs, options, and activations all become Atoms in shared memory, operating under unified scheduling. This means reasoning can directly guide where neural networks focus their attention, while neural networks can propose structured hypotheses back to the reasoning system—all without serialization boundaries or synchronization bottlenecks.

7.1 Two Paths to Integration

Hyperon offers two complementary approaches to neural-symbolic integration:

The “Outside” approach treats existing neural networks as services, wrapping them so their embeddings and activations appear as Atoms in the Atomspace. This pragmatic path lets you leverage existing models immediately—your favorite LLM or vision model can contribute to reasoning without modification.

The “Inside” approach through QuantiMORK embeds neural computations directly within MORK’s data structures. Tensors become multiresolution DAGs (using wavelet-style hierarchies), with attention mechanisms and gradient updates running on the same PathMap structures that store symbolic knowledge. MeTTa rules orchestrate these computations, deciding where refinement is needed based on uncertainty and importance.

The inside approach enables extraordinary capabilities: pattern mining can identify valuable features that immediately influence attention, symbolic proofs can constrain neural predictions, and computation can selectively expand only where it matters most. We will now elaborate a little bit on how this is envisioned to work within Hyperon—with the caveat that this is current R&D and we have not yet demonstrated every piece of the design in action at scale.

7.2 Symbolic Heads: Giving Transformers Structured Memory

7.2.1 The Problem with Pure Attention

Standard Transformers, for all their capabilities, struggle with structured knowledge. They compress everything into continuous vectors, losing track of discrete relationships, logical constraints, and reusable templates. When context windows fill or domains shift, these models often forget hard-won structural insights.

Hyperon gives us the potential to introduce Symbolic Heads, addressing these issues by augmenting each Transformer layer with a discrete memory system. Think of it as giving the model a structured notebook where it can store and retrieve templates, rules, and patterns that complement its continuous representations.

7.2.2 How Symbolic Heads Work

Our strategy for implementing symbolic heads adds surprisingly little complexity to standard Transformers:

1. **Template Library Creation:** We parse training text into Atomspace graphs and mine frequent subgraphs using WILLIAM. These become key-value pairs (p_j, v_j) in our template store—the patterns are keys for retrieval, the associated structures are values to inject.
2. **Alignment Learning:** Each layer learns to project its hidden states into a key space that aligns with template keys. A contrastive loss ensures the model learns when structural templates are relevant, while a reconstruction loss ensures retrieved values integrate meaningfully.
3. **Runtime Retrieval:** For each token, the model:
 - Projects hidden state h_i to a query q_i
 - Retrieves top-m templates by similarity
 - Computes attention over retrieved templates
 - Injects the weighted combination back into the hidden state

The beauty is that this happens in parallel with standard self-attention. The symbolic memory doesn’t replace neural processing—it augments it with structured priors that improve coherence and generalization.

7.2.3 Where the Templates Come From

The Pattern Miner provides the symbolic content, following the elegant pipeline we use throughout PRIMUS: generate abstract patterns $\rightarrow$ specialize them $\rightarrow$ filter by support $\rightarrow$ expand conjunctions $\rightarrow$ rank by I-surprisingness. Since we keep everything as lazy streams until the final scoring step, the mining process integrates naturally with the dynamic needs of the Transformer.

7.3 Pattern Mining as Neural Prior Discovery

The patterns that WILLIAM and our mining pipeline discover aren’t just useful for symbolic reasoning—they’re exactly the kind of structural priors that help neural networks generalize better. By continuously mining patterns from both the knowledge base and the neural network’s own representations, we create a virtuous cycle where structure discovery enhances both symbolic and neural processing.

The natural strategy for implementing leverages MeTTa’s stream-based processing:

- **support** counts pattern occurrences without materializing full result sets
- **match-count** efficiently tallies matches using PathMap indices
- **expand-conjunction** creates complex patterns from simple ones
- **i-surprisingness** identifies patterns that are both common and informative

These operations work lazily—we can mine patterns from trillion-edge graphs without exhaustive enumeration. When Symbolic Heads request patterns matching certain criteria, the miner can generate them on demand rather than maintaining a fixed library.

7.4 Inside Atomspace: The (More Radical) QuantiMORK Approach

7.4.1 Neural Networks as Native Graph Operations

A more interesting and original way to integrate deep neural nets with Hyperon is to actually put the neural nets in the Atomspace. Doing this naively, however, would likely result in poor performance. The QuantiMORK design works around this by focusing on specific DNN architectures that match up structurally with the MORK internals in a natural way—because they rely internally on wavelet decompositions, which involve a hierarchy that maps naturally onto MORK’s internal prefix tree structures. In QuantiMORK, instead of tensors living in separate memory that we occasionally sync with symbolic structures, they become multiresolution trees directly within the Atomspace.

7.4.2 The Technical Architecture

QuantiMORK encodes tensors using discrete wavelet transforms, storing coefficients as Atoms keyed by (level, band, position). This hierarchical representation naturally supports:

- **Selective refinement:** Compute only bands where uncertainty is high
- **Progressive processing:** Start coarse, refine where needed
- **Structure sharing:** Common subpatterns become shared subtrees

Ground-atom operations like `gpu-conv`, `gpu-loss`, and `gpu-backprop` work directly on these structures. MeTTa rules orchestrate the computation, implementing everything from standard forward passes to novel predictive coding updates that proceed locally without global backpropagation.

7.4.3 Predictive Coding Without Backprop

The scalability we believe will be achievable via QuantiMORK’s depends not only on the alignment of waveformer features with MORK’s internal prefix trees, but also on replacing global backpropagation (as used in traditional waveforms architectures) with local predictive coding updates. Each level of the wavelet hierarchy maintains local predictions and errors. Updates flow bidirectionally through local message passing rather than requiring a global backward pass.

This approach offers several advantages over alternative ways of implementing deep neural networks:

- Computation can focus on high-error regions
- Different parts of the network can update asynchronously
- Symbolic rules can intervene at any level
- Memory requirements scale with active regions, not total model size

7.5 Weakness-Based Stability for Neural Updates

7.5.1 The Challenge of Compositional Learning

Predictive coding networks juggle multiple update streams: bottom-up sensory errors, top-down predictions, lateral competition, and reward-based modifications. Without careful coordination, these updates may interfere destructively—i.e. there can be dynamics wherein what improves one

objective degrades another, leading to instability and catastrophic forgetting. With coordination, PC networks can display relatively non-entangled representations (cf Ken Stanley’s work on disentangled representations in his NEAT evolutionary learning framework), which is valuable for generalization as well as for tractable interoperation between neural and symbolic subsystems.

7.5.2 Commutativity as a Design Principle

Through our theoretical and experimental research with predictive coding networks, we have observed that part of the solution to such issues lies in the mathematical property of commutativity. If the different sorts of learning-updates occurring within a predictive coding network commute with each other (e.g. lateral updating within layers versus hierarchical top-down/bottom-up updating), then the network will often be better-behaved and more supportive of continual learning.

We can enforce this in predictive coding networks via various strategies, including the tactic of adding a non-commutativity penalty to the loss function used for training. This creates pressure toward commutative updates—not as a hard constraint but as a soft preference that maintains flexibility while improving stability.

7.5.3 Connection to MetaMo

This mathematical framework mirrors MetaMo’s design principle that appraisal and decision should approximately commute. Just as cognitive motivations should be relatively stable regardless of evaluation order, neural updates should compose predictably regardless of application sequence. The same mathematics that ensures goal stability also ensures learning stability. These are two among the multiple composing and braiding commutativity’s that characterize the updated, TransWeave-based Hyperon design.

7.6 WILLIAM-Guided Neural Efficiency

7.6.1 Compression Drives Computation

WILLIAM’s adaptive compression capability can be used for a variety of purposes. It can be used directly to compress knowledge so that more fits in RAM; it can be used as a strong AGI heuristic, because compression correlates closely with generalization capability; and it can also be used to enhance the efficiency and intelligence of other AGI methods.

As an example of the latter, WILLIAM can be used to increase the efficiency of neural nets implemented in the MORK infrastructure e.g. via AdaptiMORK. In this context WILLIAM effectively guides where neural computation should focus. By maintaining compression statistics on MORK’s trie nodes, WILLIAM can identify:

- Which attention heads contribute most to prediction
- Which tokens carry the most information
- Which frequency bands matter for current tasks
- Which model components could be pruned without loss

7.6.2 Dynamic Sparsity in Practice

The integration may work through several mechanisms:

Heavy-hitter iterators identify the most important subpatterns in real-time, without global scans. These patterns become attention targets and help prioritize computation.

Weakness-based pruning removes model components that don't sufficiently distinguish between inputs—if two different inputs produce nearly identical activations in some component, that component isn't pulling its weight.

Selective refinement expands computation only where WILLIAM's metrics indicate high value. This isn't just dropping random connections—it's principled sparsity based on information-theoretic value.

This is a strategy for speeding up MORK-based transformers (and other MORK-based neural net architectures) not through clever engineering tricks but through principled application of compression theory.

7.7 Example Neural/Hyperon Workflows

7.7.1 Workflow A: Outside Integration with Symbolic Augmentation

Let's trace how an external LLM benefits from symbolic integration:

1. **Pattern library preparation:** The miner processes your knowledge base, extracting templates scored by support and I-surprisingness.
2. **Runtime augmentation:** As the LLM processes text:
 - Each layer projects states to symbolic queries
 - Retrieves relevant templates from MORK
 - Blends symbolic and neural information
 - Continues with enriched representations
3. **Stabilization:** Weakness penalties and commutativity regularization ensure the symbolic additions don't destabilize the pre-trained model.

This workflow requires minimal modification to existing models while providing substantial benefits in coherence and factual grounding.

7.7.2 Workflow B: Inside Integration with QuantiMORK

For deeper integration, consider a predictive coding network living entirely within Atomspace:

1. **Initialization:** WAS selects active regions based on importance. ByteFlow maps these to kernel-friendly memory layouts.
2. **Computation:** Waveformer kernels process the multiresolution representation:
 - Intra-scale operations handle local interactions
 - Sparse cross-scale fusion connects levels
 - PC updates flow locally without global backprop
3. **Pattern integration:** WILLIAM identifies emerging patterns in network activations. These become:
 - New Symbolic Head templates

- PLN inference rules
 - Attention guidance signals
4. **Orchestration:** MeTTa rules coordinate the entire process, deciding where to refine, when to consolidate, and how to balance symbolic and neural contributions.

7.8 The Deeper Significance

What is important to realize here is: This isn't just bolting symbols onto neural networks or adding some neural processing to symbolic systems. By sharing the same memory substrate, evaluation metrics, and control mechanisms, neural and symbolic processing become complementary aspects of a single cognitive process.

Patterns discovered symbolically become neural priors through Symbolic Heads. Neural activations become hypotheses for symbolic reasoning through QuantiMORK. Both pay the same simplicity tax through weakness regularization. Both follow the same resource allocation through geodesic control.

The result is a system where improvements in pattern mining immediately benefit neural attention, where neural discoveries immediately become available for logical reasoning, and where the boundary between symbolic and neural computation becomes a matter of implementation convenience rather than architectural constraint.

This unified approach suggests that the historical divide between symbolic and connectionist AI may have been an artifact of implementation limitations rather than a fundamental requirement. When both approaches share the same substrate and speak the same mathematical language, they naturally enhance rather than compete with each other.

8 Planning for the AGI → ASI Transition: Stability, Reflection, and Safe Self-Modification

8.1 The Challenge of Ongoing Radical Self-Improvement

Creating a capable AGI is both the end of one amazing journey and the beginning of another. A core part of the AGI challenge—and opportunity—lies in building a powerfully generally intelligent system that can improve itself without suddenly or erratically losing its purpose or values. This may at first sound like hubris or science fiction, but after more reflection it becomes clear this is actually more of a practical necessity. Any AGI worth building will need, want and endeavor to upgrade its capabilities, refine its algorithms, and scale its processing. The crucial question is: how can it do this while remaining aligned with human values and maintaining its core objectives?

Traditional software can be upgraded by external developers who understand its purpose. But an AGI that truly deserves the name must be able to modify itself, and those modifications must preserve the properties we care about – or at least, they must do so initially, and deviate from this course only after an appropriate level of care and deep reflection. This requires three interconnected capabilities:

1. **Goal preservation** - A way to represent and maintain objectives that survive through system upgrades
2. **Compositional safety** - Shared principles that ensure modules work together predictably even as they evolve

3. **Controlled modification** - A procedure for self-improvement that includes validation, testing, and rollback

8.1.1 The Technical Framework

We address these challenges in the current Hyperon design through a combination of mathematical frameworks and engineering practices. Goal stability becomes a multi-level fixed-point problem with formal convergence guarantees. Global regulators—geodesic effort and weakness-based simplicity—appear throughout the system, ensuring consistent behavior across modifications. The self-modification pipeline treats changes as typed transformations with admission certificates, simulation testing, and capability-controlled deployment.

8.2 Goal Stability as a Tractable Math Problem

Goals aren’t monolithic—they’re layered structures where higher levels constrain how lower levels can evolve. Consider human values: we have immediate goals (finish this task), persistent objectives (maintain health), core values (preserve human dignity), and meta-values about how values themselves should change (remain open to growth while preserving identity). The relation between the goals on different levels is not purely top-down nor bottom-up but is more like the relationship between the layers in a predictive-coding neural network: ongoing iterated inter-modification leading to holistic self-organization of the network across all levels.

An AGI needs the same kind of hierarchical goal structure. Each level acts as an invariant that constrains acceptable changes to the level below. This creates a “stack” of purposes where evolution is possible but bounded.

8.2.1 The Mathematics of Stability

We formalize this through invariant hierarchies. Goals become a family $I = \{I_1, I_2, \dots\}$ where each I_k constrains how I_{k-1} can evolve. These might be sets of value functions, acceptance regions in policy space, or formal contracts about subsystem behavior.

Self-modification becomes an operator T acting on the pair (mechanism, invariants). We seek fixed points or stable attractors where $T(\text{mech}, I) \approx (\text{mech}', I')$ with I' remaining within acceptable bounds of I .

The mathematics provides two types of guarantees:

- **Strong stability** emerges when T is contractive in an appropriate metric (like bisimulation distance for policies or f-divergences for value distributions). The Banach fixed-point theorem then guarantees a unique stable configuration. This situation will occur when the system’s self-modification intentionally makes modifications according to some core set of principles or playbook.
- **Weak stability** applies when we have continuity but not contraction. Schauder-type theorems guarantee that stable points exist, though they might not be unique. In these cases, we add monitoring and drift budgets to ensure the system stays within acceptable bounds. This situation will occur when there is a bound to the complexity that the system will self-modify into, and some continuity to its pattern of self-modification – a more open-ended sort of self-modification that is not necessarily compatible with stronger forms of stability.

8.3 Global Regulators: Consistent Principles Across All Cognition

Throughout the new incarnation of PRIMUS, two mathematical principles guide every decision:

Geodesic effort ensures that all processes—*inference, learning, planning, and even self-modification*—follow minimum-effort paths that balance forward progress (what’s reachable) with backward pull (what’s valuable). This creates a natural efficiency that prevents wasteful exploration while maintaining goal-directed progress.

Weakness-based simplicity provides a universal preference for simpler solutions. Whether choosing between proof strategies, neural architectures, or system modifications, the weakness measure quantifies “failure to distinguish” and creates consistent pressure toward more general, robust solutions.

If these two principles are leveraged to militate toward moderated and beneficial goal evolution, then this will permeate through all the particular algorithms and structures used throughout the system. Seeking the simplest possible representations of system goals, and using these to follow shortest-paths toward futures where the goals are substantially realized, gives PRIMUS systems a stronger chance of maintaining ethical behaviors even as they self-modify.

This is an approach in which increased capability doesn’t mean decreased predictability. The system becomes more powerful while remaining comprehensible and controllable.

8.4 The Self-Modification Pipeline

8.4.1 Beyond Hot-Patching

Self-improvement isn’t about making quick fixes and hoping for the best. We’ve developed a rigorous five-stage pipeline that treats each modification as a formal experiment:

1. **Proposal** - Generate a candidate change (new algorithm, data structure, or module)
2. **Analysis** - Examine theoretical impact through type checking and dependency analysis
3. **Simulation** - Test in a digital twin environment with reduced but representative workloads
4. **Certification** - Verify that invariants are preserved and performance goals are met
5. **Deployment** - Stage rollout with monitoring, capability restrictions, and rollback options

8.4.2 Technical Details

Each modification is formalized as a typed metamorphism $\theta : C \rightarrow C$ over the category of system components. The type signature includes:

- Pre- and post-conditions that must be satisfied
- Promised improvements in complexity or performance
- Expected changes in weakness (simplicity) metrics

Static analysis builds an influence graph showing which components are affected. The system checks lens laws to ensure updates compose properly and validates that the modification preserves essential structural properties.

Twin simulation creates a controlled copy of relevant system components. Using Weighted Atom Sweeps to select representative regions, the system replays realistic workloads while measuring:

- Value function divergence from original behavior
- Weakness trajectory (is it becoming simpler or more complex?)
- Commutativity penalties (for neural components)
- Evidence conservation (no hallucination or information loss)

Certification requires meeting all safety criteria:

- Invariant levels I_k remain within their prescribed bands
- Behavioral drift stays below threshold: $\text{div}(\text{old}, \text{new}) \leq \varepsilon$
- Weakness improves or remains neutral
- Evidence flow maintains conservation properties

The system produces a formal admission certificate documenting what transfers successfully, what must be relearned, and what monitoring is required.

Staged deployment uses RSpace/Rholang executors with capability-based security. Changes initially run in shadow mode alongside the original, graduate to dual-run with comparison, then elevate to primary when metrics stabilize. All artifacts—patches, certificates, metrics—are content-addressed for complete audit trails.

8.5 Safety Through Transparency

8.5.1 Making Safety Checkable

Every component and modification carries explicit safety documentation that higher-level processes can verify:

Type interfaces specify behavioral contracts including domain/range constraints, effect bounds (latency, memory usage), and monotonicity properties where relevant.

Admission certificates come in two varieties:

- SubRep certificates validate that new options genuinely serve declared goals
- TransWeave certificates document successful component transfers, weakening factors, and regions requiring relearning

Drift budgets establish tolerance bands for each invariant level. The system maintains supermartingale potentials (Lyapunov-like functions) that must decrease in expectation. If potentials rise, the scheduler automatically reduces step sizes or triggers rollback.

8.6 Decentralized Governance

8.6.1 Multi-Party Evolution

Real-world deployment often involves multiple teams, organizations, or even multiple AI systems collaborating on improvements. Hyperon’s architecture supports this through:

Capability security - Self-modification jobs compile to Rholang processes running under RSpace with object-capability guards. Edit permissions are precisely scoped to specific Spaces, prefix ranges, or component classes.

Cryptographic provenance - Every modification artifact receives a content identifier (CID). The FireNode/F1R3FLY executor layer posts summaries to chain, creating an immutable audit trail of who changed what, when, and why.

Atomic recovery - ShardZipper’s Merkle-based structure enables instant rollback to any previous state. The State Management System maintains macro-patches that bound recovery time and provide clear explanations of what changed.

8.7 A Concrete Example: Upgrading Predictive Coding

8.7.1 Walking Through the Pipeline

Let’s trace how the system might upgrade a predictive coding component to use WILLIAM-guided sparsity:

Proposal: Replace dense attention with top-k selection guided by compression metrics. Promised benefits: 30% fewer FLOPs, $\leq 2\%$ accuracy loss.

Analysis: The influence graph shows dependencies on pattern-head calls, PLN hooks, and mining iterators. Lens laws verify that ByteFlow patch logs and ShardZipper operations remain valid.

Simulation: The system clones hot prefixes identified by WAS, then runs the modified component with 25% geodesic budget. Monitoring tracks:

- Value drift using bisimulation metrics
- Weakness trajectory showing improved simplicity
- Commutativity penalties remaining low

Certification: Results show all options in domains A and B remain admissible (SubRep validation). Two H-ICA components transfer cleanly, one requires relearning (TransWeave analysis). Drift stays below ε threshold. The supermartingale potentials Φ_1 and Φ_2 decrease as required.

Deployment: The upgrade runs in shadow mode for 24 hours, graduates to dual-run with comparison, then elevates to primary when KPIs stabilize. The last-good CID remains available for instant rollback if needed.

The result: faster, simpler predictive coding that maintains all safety properties and preserves downstream behavior in proof and planning systems.

8.8 Implementation Checklist

8.8.1 Building Self-Improvement Capabilities

To implement safe self-modification:

Goal representation: Build a library for invariant hierarchies with support for multiple metric families (bisimulation, f-divergences, weakness). Expose `check_invariant(k, bands, sample)` as a grounded operation.

Geodesic control: Create a shared service that meters effort across all cognitive operations. Provide `even_cost_step( $\Delta \log f$ ,  $\Delta \log g$ , cost)` to ensure consistent resource allocation.

Weakness tracking: Store $Q_{\text{logic}} \times Q_{\text{tv}}$ valuations with all Atoms. Make per-edge weakness deltas available to every scheduler. Use weakness as the universal regularizer.

Modification pipeline: Implement:

- Typed edit proposals with lens checking

- Influence graph construction
- Twin environment creation and management
- Divergence and commutativity meters
- Certificate generation
- Staged rollout orchestration

Decentralized execution: Create Rholang templates for capability-controlled modifications, FireNode hooks for posting certificates, and SMS integration for reproducible rollback.

Monitoring dashboards: Build live displays for supermartingale potentials, divergence bands, weakness trends, and geodesic budget consumption—with one-click hold/elevate/rollback controls.

8.9 The Core Insight

8.9.1 Unity of Principles

The profound insight underlying this approach is that the same mathematics that governs routine cognition can also govern self-modification. If daily reasoning follows geodesic paths, so should system upgrades. If simplicity (weakness) helps generalization in learning, it should also guide architectural evolution. If goal decomposition through MetaMo/SubRep works for planning, it should also work for validating new capabilities.

This isn't just philosophical elegance—it's practical engineering. By using the same frameworks throughout, we ensure that improvements are improvements in the fullest sense: the system becomes more capable while remaining aligned, more powerful while staying comprehensible, more autonomous while preserving human values.

The path from AGI to ASI isn't a discontinuous leap into the unknown. It's a sequence of principled, verified, reversible steps, each preserving what matters while extending what's possible. The mathematics gives us confidence; the engineering gives us control; and the unified framework ensures that as the system grows more capable, it also grows more trustworthy.

9 Example Application Domains: From Current Pilots to General Intelligence

Our development philosophy is straightforward: build real applications in diverse domains from day one, using each as both a testbed and a source of learning that benefits all others. We currently have working pilots in four key areas—game AI, humanoid social robotics, bioinformatics hypothesis generation, and mathematical conjecturing with formal proof. While each application starts simple, they all connect to the same Hyperon+PRIMUS substrate, meaning advances in perception, reasoning, pattern mining, motivation, and transfer benefit every domain simultaneously. The path to general intelligence isn't through four separate technology stacks but through one unified system whose capabilities are exercised and evolved across radically different challenges.

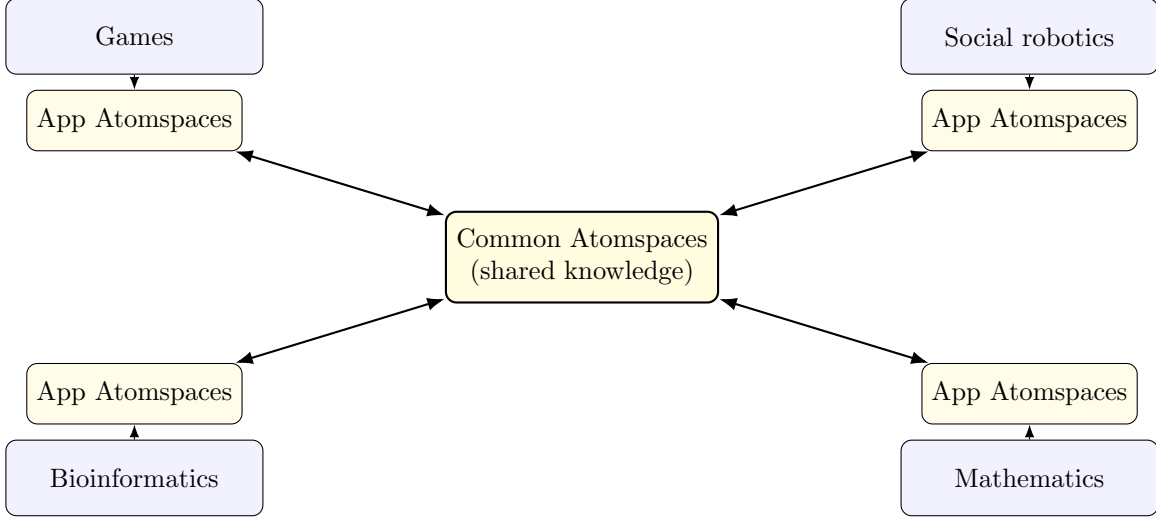


Figure 4: Multiple applications share common "collective knowledge" Atomspaces while maintaining local per-application Atomspaces for task-specific processing.

9.1 Game AI: From Minecraft to Sophiaverse and the Neoterics Playground

9.1.1 The Opportunity

Games provide ideal sandboxes for testing perception, planning, tool use, dialogue, and social norms with rapid iteration and safe failure. Minecraft offers a structured yet open world perfect for integrating SubRep options with GEO-EVO edits. Sophiaverse adds persistent identity, social contracts, and economic systems. Within Sophiaverse, we’re designing a specialized “Neoterics” micro-world—a deliberately constrained but richly instrumented space optimized for baby-AGI development, featuring short feedback loops, dense sensor coverage, and cheap resets that allow PRIMUS components to be trained and tested rapidly.

9.1.2 Technical Implementation

The world interface operates through Spaces, mirroring game state (voxels, entities, quests, markets) into Atomspace as typed Atoms. Action affordances appear as SubRep options (navigate, craft, trade, negotiate) with MeTTa rules expressing pre/post-conditions. Each option admission receives certification (CDS/PDS) and storage with CIDs for full auditability.

For perception and policy, we initially use external LLMs for narration and quest generation, with Symbolic Heads retrieving mined templates like task frames, recipes, and negotiation scripts. Inside the graph, QuantiMORK hosts lightweight PC waveformers that locally refine spatial or dialogue embeddings. WILLIAM steers attention by managing top-k heads, tokens, and scales while prioritizing unification order based on heavy-edge importance.

Reasoning operates through PLN factor-graphs that encode precondition networks and multi-step plans. Geodesic control ($f \cdot g$) scores candidate steps by combining reachability gains with usefulness gains per unit cost—the same selection rule governing MOSES edits in programmatic policies.

Learning and transfer work through GEO-EVO, which couples MOSES demes (for crafting plans and trading tactics) with backward goal factors. TransWeave carries successful options and

value maps from Minecraft to Sophiaverse/Neoterics when H-ICA component matches prove strong, with weakness gates preventing brittle overfitting.

The runtime leverages Weighted Atom Sweeps to maintain salience, ByteFlow PPTNs to pack hot subtrees for GPU kernels, and ShardZipper to batch world-tick updates deterministically. Consider a micro-flow in Neoterics for “build-and-barter” gameplay: The pattern miner discovers the motif $\langle \text{mine} \rightarrow \text{smelt} \rightarrow \text{craft} \rightarrow \text{trade} \rangle$. PLN assembles a three-step plan. Geodesic control selects the cheapest next step. The PC waveformer refines local perception only where surprise exceeds threshold. WILLIAM prunes attention heads and tokens for efficiency. Upon successful completion, TransWeave logs an option map that Sophiaverse agents can reuse with adjusted prices.

Near-term milestones include a Symbolic Head MVP for quest/recipe frames with weakness-aware pruning, PLN + SubRep integration for 5-10 canonical Minecraft tasks with GEO-EVO corridor demonstrations, and a Neoterics “micro-agency” loop achieving sub-second decisions with PC local refinement and WAS scheduling.

9.2 Humanoid Social Robotics: Education and Performance

9.2.1 The Partners and Vision

We’re working with two pioneering robotics companies. Mind Children is developing a 3.5-foot tall education robot designed for classroom interaction. Hanson Robotics, creators of Sophia (the world’s first robot citizen) and Desdemona (the robot singer), brings expertise in expressive humanoid robots for public engagement. These partnerships provide the perfect stress-test for PRIMUS: robots must combine perception, dialogue, motor control, and social norms seamlessly. The cognitive substrate must handle everything from “explain fractions to a third-grader” to “comfort an anxious student” to “perform on stage”—without requiring a complete rewrite for each scenario.

9.2.2 Technical Architecture

Perception begins with audio and vision neural networks running externally, but we progressively embed PC layers inside via QuantiMORK for latency-critical paths like gaze estimation, lip-sync, and gesture segmentation. Prediction errors feed PLN hypotheses (“student is pointing at X”, “person is asking a question”), while symbolic constraints steer refinements, expanding only the frequency bands tied to current hypotheses.

For dialogue and social scripts, Pattern Miner discovers conversational templates including adjacency pairs and tutoring frames. Symbolic Heads allow the speech model to retrieve these patterns during decoding, while PLN enforces persona and safety constraints. MetaMo budgets the balance between exploration and de-escalation behaviors, throttling novelty when emotional risk is high.

Motor control keeps low-level servo loops on the robot while SubRep options cover higher-level skills (“look at face”, “nod”, “hand-wave”). Geodesic control aligns dialogue steps with motion options, maximizing $\Delta(\log f + \log g)/\text{cost}$ so physical gestures support conversational goals. Weakness penalties prevent over-committed or awkward gestures.

Safety and alignment leverage Meta-Goal Stability to track drift in social norms and persona. Commutativity penalties ensure PC “sensation” updates and RL “decision” updates don’t conflict. RSpace executors and lens contracts strictly gate what an on-device agent may modify.

The runtime uses WAS for background maintenance (belief cleanup, memory decay), ShardZipper to extract “interaction shards” (short temporal windows) for batch prosody/gesture retiming, and ByteFlow to handle drifting hot regions in live scenes.

A typical classroom tutoring micro-flow: Student asks a question. ASR converts to PC features, generating PLN hypotheses about intent. Symbolic Head retrieves the appropriate tutoring frame. MetaMo selects an “explain + gesture” option bundle. Geodesic step executes “point to board then speak”. PC refines lip/gaze locally for natural interaction. WILLIAM prunes unused heads for efficiency. Capsule logs capture evidence to prevent double counting.

Near-term milestones include inside-graph PC for gaze/lip/speech synchronization with commutativity regularization, a SubRep library of 20-30 certified gesture/dialogue options with admission certificates, and a safety dashboard showing invariant bands for persona/etiquette, drift budgets, and rollback controls.

9.3 Bioinformatics: Graph-Based Discovery for Longevity and Beyond

9.3.1 Why Hyperon Excels Here

Biology is fundamentally structured as graphs: genes connect to proteins, proteins form pathways, pathways influence phenotypes, drugs modulate these relationships. Longevity research isn’t about blindly processing every dataset but about forming testable hypotheses—identifying targets, combinations, and mechanisms worth experimental validation. Hyperon excels at combining noisy graphs, mining meaningful motifs, running uncertain chains of reasoning, and proposing ranked, testable hypotheses with clear rationales for prioritization.

9.3.2 System Architecture

Data flows into Atomspace through BioSpace adapters that transform omics matrices into node attributes, protein-protein interactions and pathways into edges, literature triples into assertions with provenance, and clinical outcomes into noisy links with confidence scores. Everything receives CIDs, making merges auditable and reproducible.

Pattern discovery operates through Pattern Miner, which identifies motifs like “gene A $\leftrightarrow$ pathway P $\leftrightarrow$ phenotype Y with drug D evidence”, ranking them by I-surprisingness and support. WILLIAM transforms frequent subgraphs into reusable templates representing candidate mechanisms.

Reasoning and causality flow through PLN factor-graphs that propagate graded truth over ontologies, experimental results, and pathway priors. AIRIS-style causal induction constraints filter implausible relations. A geodesic $f \cdot g$ planner assembles mechanism candidates by balancing reachability from data with usefulness for therapeutic objectives, maintaining constant-effort steps throughout.

For programs and predictions, MOSES/GEO-EVO evolves small predictive programs (risk scores, treatment effect surrogates) guided by SubRep subgoals like “improve precision on aged cohort”. Weakness (Occam’s razor) controls model complexity, with models stored as MORK objects including evaluation capsules.

Transfer operates through TransWeave, which moves mechanism components across cohorts or omics platforms when H-ICA matches hold strong. Impossibility flags force partial transfer or fresh learning when appropriate, preventing brittle reuse.

The runtime uses ShardZipper to extract cohorts or pathway-centric shards, ByteFlow to accelerate repeated motif scans, and WAS to schedule refresh where new data arrives.

Consider an example combo therapy hypothesis micro-flow: The miner highlights a motif linking mTOR and autophagy via two drug actions. PLN infers a plausible double-hit mechanism under aging markers. MOSES proposes a response predictor. The geodesic planner selects the next assay step optimizing $\Delta(\log f \cdot g)/\text{cost}$. TransWeave pulls in matched components from related datasets.

The output is a ranked hypothesis pack (CID bundle containing mechanism graph, predictor, expected biomarkers, and counter-evidence).

Near-term milestones include an end-to-end hypothesis generator on a large longevity dataset combining omics and literature, GEO-EVO corridor with $f \cdot g$ guidance for experiment selection and SubRep certificates for assay steps, and a weakness-aware model portfolio emphasizing few, simple, transferable models with full audit trails.

9.4 Mathematical Theorem Proving with Automated Conjecturing

9.4.1 Beyond Proof to Discovery

Most automated theorem proving focuses on proving stated goals, but mathematicians spend much of their time on conjectures—formulating new definitions, lemmas, and “what if” hypotheses that later get proved, refuted, or refined. We’re targeting automated conjecturing as the primary goal, then using formal proof for validation. We’ve recently implemented a formal proof verifier in MM2 inside MORK, giving us a fast, graph-local proof kernel that operates at native speeds.

9.4.2 Technical Design

Key components:

- The knowledge base stores definitions, theorems, proofs, and proof tactics as Atoms. A MathSpace wrapper handles import/export to external proof assistants while maintaining CIDs and equivalence classes (normal forms) internally.
- Pattern discovery through Pattern Miner finds proof motifs (induction schemas, commutativity tricks) ranked by I-surprisingness. WILLIAM promotes frequent motifs to templates usable by Symbolic Heads for tactic prediction and by PLN as proof search heuristics.
- The conjecture engine treats conjecturing as geodesic search over a space of typed expressions. Forward factors (f) encode reachability from known lemmas, while backward factors (g) encode usefulness (closing gaps, unifying motifs). Weakness biases toward simple, general statements. TransWeave reuses isomorphic substructures from neighboring theories (for example, transferring group theory lemmas to ring theory).
- For proof and verification, candidate conjectures enter a pipeline: first attempting schematic proofs with PLN hints, then calling external provers when helpful, validating in the MM2 proof verifier on MORK for fast local checking, and finally storing results and counterexamples with CID provenance.
- The runtime keeps hot regions (active theories) fresh through WAS, uses ShardZipper to extract theory shards for batch tactic synthesis, and leverages ByteFlow for rapid local normalizations and set-algebra over large term DAGs.

An algebraic identity micro-flow based on these components looks something like: The miner spots recurring curvature in proof trees. WILLIAM promotes it as a pattern. The conjecture engine proposes a parametric lemma generalizing it. PLN provides a skeleton proof. MM2 verifier checks validity. On failure, the system generates a refined conjecture with weaker premises. The final result becomes a template in the library and a TransWeave candidate for sibling theories.

Near-term milestones along this path include exposing the MM2 proof verifier as a MeTTa ground operation with CIDs and patch logs, developing a conjecture engine MVP using $f/g +$

weakness + miner templates on two small theories with an external prover bridge, and integrating Symbolic Head for tactic suggestion with an external model backed by internal templates.

9.5 Why These Domains Matter Together

9.5.1 Shared Infrastructure Wins

The same priors operate everywhere. Weakness governs simplicity in game policies, social scripts, biological mechanisms, and mathematical conjectures—creating a global bias toward simpler adequate solutions that also improves transfer.

The same control law applies universally. Geodesic ($f \cdot g$) balances reachability and usefulness whether crafting tools, coordinating gestures, selecting assays, or extending lemmas—making edit/step selection principled and comparable across all domains.

The same provenance and safety mechanisms work everywhere. CIDs, patch logs, capsules, ocap-guarded deployments, and invariant bands provide auditability whether the agent trades in Sophiaverse, talks to a child, ranks a drug target, or proposes a lemma.

The same neurosymbolic loop drives progress. The Pattern Miner $\rightarrow$ WILLIAM $\rightarrow$ Symbolic Heads/PLN cycle operates identically across domains: patterns discovered symbolically become neural priors, neural activations generate hypotheses, and proofs/plans flow back into mining—always within one unified memory and control plane.

The beauty of this approach is that each domain provides external, human-meaningful validation: survival and health metrics in medicine, learning outcomes in education, theorem verification in mathematics, and engaging gameplay in virtual worlds. These aren’t just test cases—they’re shaping forces that build the agent’s fundamental priors around evidence-based explanations, even-cost progress, simple/general representations, and respectful social interaction. As these priors strengthen through diverse experience, they naturally influence behavior everywhere, because the same mathematical framework (weakness, geodesic control) operates uniformly across the entire system.

10 Why This Is a Path to Beneficial AGI

The concept of “beneficial” AGI encompasses three essential qualities: the system must be capable (able to learn and reason across modalities and tasks), aligned (carrying explicit, inspectable models of its own motives and constraints with the ability to reflect on changes), and accountable (deployed under verifiable rules with transparency). These aren’t aspirational goals—they’re operational requirements built into the architecture from the ground up.

Hyperon + PRIMUS represents a practical route to beneficial AGI through three mutually reinforcing design pillars that work together as a coherent whole.

10.1 Reflective Motivation and Self-Modification

10.1.1 The Gap in Current Systems

Most AI systems today, including large language models, can discuss values and suggest ethical principles, but they don’t actually operate according to values in any meaningful computational sense. They also lack any internal mechanism for understanding or explaining why their behavior changes over time. This creates a fundamental opacity: we can observe what they do, but not why they do it or how they might change.

Hyperon takes a different approach by installing an explicit motivation layer (MetaMo) that sits above perception, learning, and planning components. This gives self-modification a formal admission process with clear boundaries. The system can articulate what it’s optimizing for at any moment, explain the trade-offs it’s considering, modify itself within specified guardrails, and—crucially—prove that it stayed within those guardrails.

10.1.2 How It Works Technically

The MetaMo system implements motivation through coupled appraisal and decision functions. Appraisal (Ψ) evaluates “how I assess this situation” while Decision (D) determines “what I will do about it.” These connect via a lax distributive law, creating a pseudo-bimonad $F = D \circ \Psi$ that allows the agent to compute these in either order with bounded discrepancy. Every appraisal/decision trace gets logged alongside the actions taken, creating an auditable record of motivational reasoning.

Skills and options (whether plan fragments, dialogue scripts, or lab procedures) require admission through SubRep certificates. These CDS/PDS certificates explicitly state preconditions and safety/ethical constraints. When options are chained together, composition rules ensure the certificates remain valid throughout the sequence.

The system selects actions using geodesic control, which maximizes $\Delta(\log f + \log g)/\text{cost}$, where f represents reachability (what the world allows) and g represents usefulness (what our values prefer), with constant per-step effort to prevent reckless leaps. Importantly, this same scoring mechanism governs both regular actions and self-modification edits, creating consistency between operation and evolution.

A single quantale-valued weakness prior acts as a universal simplicity bias, penalizing brittle or over-committed structures across all modules—*inference* (PLN), *program search* (MOSES/GEO-EVO), *neural updates* (predictive coding commutativity), and *system edits*. This unified regularization ensures different components don’t pull in incompatible directions.

Goal stability is maintained through mathematical invariants $I_1, I_2, \dots$ that constrain how goals can evolve, including meta-goals about goals. Self-modification operators must demonstrate either contraction (strong stability) or at least continuity on compact domains (weak stability). The runtime continuously measures drift budgets and exposes rollback triggers when supermartingale functions Φ_k (one per invariant level) exceed thresholds.

The self-modification pipeline ensures safety through multiple stages: edits are typed and simulated in a digital twin with bounded geodesic budget, then certified (checking invariants stay within bands, evidence is conserved, weakness doesn’t degrade), and finally staged under capability security with CID-addressable patch logs and live rollback capabilities.

10.1.3 Why This Promotes Beneficial Behavior

Using the same calculus for both day-to-day actions and self-modification creates coherent rather than adversarial pressures between optimization and safety. Because motives, certificates, budgets, and edits exist as objects in Atomspace, they become introspectable, auditable, and transferable across teams and deployments. The system doesn’t just have goals—it can reason about them.

10.2 Decentralized Implementation on F1R3FLY/ASI-Chain

10.2.1 The Governance Problem

When a small group controls an AI system—able to switch it off, redirect it, or restrict access—the incentives inevitably skew away from public benefit toward private advantage. A truly beneficial

AGI needs not only the right motivational system and a good education, but also a plural, verifiable runtime infrastructure where communities can run, inspect, and contribute to the same cognitive stack without single points of control or opaque policy changes. This isn't governance by committee or promise—it's governance by mathematical design.

10.2.2 The Technical Solution

The Hyperon system achieves decentralization through several interlocking mechanisms:

Content-addressed everything: Knowledge, models, proof steps, motives, edits, and certificates all become CID-addressed objects with Merkle proofs. The question “what changed and why?” always has a cryptographically verifiable answer.

Capability security (ocaps): MeTTa-IL bundles compile to Rholang processes running under RSpace with least-privilege capabilities. Each process specifies exactly which Spaces it can access, which prefixes it can modify, which kernel classes it can invoke, and which write operations it's permitted.

FireNode/F1R3FLY executors: Nodes execute IL bundles and post proof-of-execution summaries (hashes, metrics, certificates) to ASI-Chain, creating an immutable audit trail. Shadow→elevate deployments and one-click rollback are protocol-level features, not ad-hoc operations requiring special privileges.

Plural governance hooks: Because CIDs and certificates are first-class objects, DAOs or cooperatives can require prior publication of certificates and KPIs before upgrades. Community validators can simulate bundles against public twins. Slashing-like penalties or reputation gating can discourage unsafe rollouts through economic incentives rather than central control.

Economic neutrality: Execution markets remain competitive with multiple providers. Data and model artifacts stay portable through CIDs. “Attention” resources (Weighted Atom Sweeps) are metered by transparent policies rather than proprietary algorithms.

10.2.3 Why Decentralization Enables Benefit

Decentralization doesn't just avoid monopoly risk and single-policy lock-in—it creates positive incentives for safety and helpfulness. Communities can tie eligibility for compute, data, or revenue to open-standard safety signals: certified motives, bounded drift metrics, evidence conservation proofs, and fairness audits. This creates market pressure for beneficial behavior rather than just raw capability.

10.3 Beneficial Applications as Training Ground

10.3.1 Shaping Through Practice

There's a fundamental principle at work here: what a mind does shapes what it becomes. By training and deploying the system primarily in pro-social, testable domains, we build cooperative patterns into its basic habits. These include providing explanations backed by evidence, prioritizing long-term outcomes over short-term gains, maintaining respectful dialogue, and producing reproducible scientific results. Because Hyperon stores the work itself—proofs, motifs, certificates—as atoms, these habits don't just influence behavior; they become reusable components across contexts.

10.3.2 Domain-Specific Implementation

Medicine and longevity discovery: Biological knowledge naturally forms graphs that Hyperon can mine for recurring mechanisms. The system runs PLN over factor graphs with uncertainty quan-

tification, proposing ranked, testable hypotheses complete with expected biomarkers and counter-evidence. SubRep certificates encode ethical and experimental constraints. Geodesic control selects measurements that maximize expected information per cost, teaching the system to value efficient, ethical investigation.

Education and social robotics: Pattern-headed speech models combined with PLN persona/safety constraints encourage transparent, patient explanations and pro-social classroom behaviors. MetaMo throttles novelty when emotions run high, while SubRep options carry etiquette and safety certificates. Every interaction remains auditable—what was said, why, and how the choice was made—reinforcing accountability as a default behavior.

Scientific discovery: The conjecture engine treats novelty and simplicity (weakness) as first-class values, proposing general statements, seeking minimal premises, and checking results with a formal proof kernel (MM2 on MORK). This keeps intellectual curiosity grounded in verifiable progress rather than speculation.

Cross-domain transfer: TransWeave moves certified structures between domains only when component matches are strong (validated through H-ICA analysis). Impossibility flags prevent brittle transfer, while weakness ensures new structures remain simple enough to compose and audit.

10.3.3 Why These Applications Matter

Each domain provides external, human-meaningful validation targets: survival and health metrics in medicine, learning outcomes and safety records in education, and theorem verification in mathematics. These aren't arbitrary benchmarks—they're shaping forces that build fundamental priors around evidence-based reasoning, preference for even-cost progress, simple and general representations, and respectful social interaction.

As the system strengthens these priors through diverse experience, they naturally influence behavior everywhere, because the same mathematical framework (weakness regularization, geodesic control) operates uniformly throughout. The habits learned in a classroom transfer to scientific reasoning; the evidence standards from medicine influence game-playing strategies; the simplicity bias from mathematics shapes social dialogue.

10.4 The Self-Reinforcing Loop

These three pillars create a virtuous cycle:

Reflective motive control (through MetaMo, SubRep, geodesic control, and weakness regularization) keeps both daily decisions and self-modifications within auditable bounds, preventing drift while enabling growth.

Decentralized execution distributes authority and requires publication of verifiable artifacts (certificates, invariants, patch logs) to access resources or trust, creating market incentives for transparency and safety.

Beneficial workloads continuously enrich the system's experience with cooperative, testable problem-solving patterns—medicine's causal rigor, education's patient dialogue, science's formal proofs—storing these as reusable atoms that shape future behavior.

10.4.1 Concrete Safeguards

The system implements multiple layers of protection by construction: capability scopes and lens contracts restrict access, invariant bands with drift budgets prevent runaway changes, evidence conservation via capsules maintains reasoning integrity, constant-effort steps prevent sudden leaps,

weakness regularization works uniformly across modules, and DAOs can gate upgrades based on published certificate bundles.

10.4.2 Measurable Success

We can track beneficial behavior through concrete metrics:

- **Alignment and process:** Monitor invariant supermartingales, commutativity penalties in predictive coding, weakness trajectories, and the percentage of actions backed by evidence capsules.
- **Governance:** Measure the percentage of upgrades shipped with twin simulation and public certificates, rollback mean time to recovery, and cross-tenant lens violations (which should approach zero).
- **Applications:** Track prospective replication rates in biology, learning gains and safety incidents in education/robotics, and conjecture-to-theorem ratios with minimality scores in mathematics.

10.5 Addressing Common Concerns

“Decentralization could still converge to oligopoly.” This is true if standards are weak. That’s why the design enforces open, mechanical signals (CIDs, certificates, KPIs) as preconditions for resources. Anyone can validate them; no one can fake them without detection.

“Adding motivation increases complexity and failure modes.” This is why MetaMo is typed and contractive, SubRep uses admission certificates, edits follow the same geodesic/weakness rules as proofs and plans, and deployment goes through twins with rollback capabilities. The complexity is managed, not ignored.

“Beneficial applications won’t prevent worst-case misuse.” They won’t eliminate all risks, but capability security and provenance systems address misuse directly. What beneficial applications do is shape the system’s priors and behavioral routines toward cooperative, testable patterns while providing constant, rich feedback aligned with public good.

10.5.1 The Bottom Line

Beneficial AGI requires thoughtful co-design of mind, machine, and milieu. We need a mind that can describe and modify its motives under proof-like constraints, a machine that many parties can run and verify without central control, and a milieu of tasks where doing good is precisely the same as doing good work.

Hyperon + PRIMUS on F1R3FLY/ASI-Chain operationalizes this co-design. The same mathematics—geodesic effort, weakness, certificates, invariants—that makes the system intelligent also makes it safe to grow. The same provenance that makes it accountable makes it democratic to deploy. The same applications that test its capabilities shape its values. This isn’t AGI with safety bolted on afterward; it’s AGI that is beneficial by construction.

11 Summary & Conclusion: A Viable, Faster Path to Beneficial AGI

11.1 Why This Path Is Plausible

At its core, Hyperon + PRIMUS represents a unified neurosymbolic machine that learns, reasons, plans, and reflects within the same memory and control plane. It can explain and audit itself as it grows—tracking motives, proofs, edits, and evidence transparently. And it’s designed for decentralized deployment, enabling many communities to help steer and verify its development. This combination makes it a credible alternative to the current paradigm of “ever-larger LLMs trained with ever-more backpropagation,” while potentially offering a faster route to human-level AGI that can scale into beneficial superintelligence.

The limitations of the LLM-only approach are becoming clear. While LLMs excel as pattern learners, they struggle with explicit long-range structure and reuse. They lack mechanisms for stable self-modification and goal control. Their reasoning processes remain opaque, making auditing nearly impossible. And their deployment model tends toward centralized, monopolistic control. Hyperon addresses each of these limitations by design rather than as an afterthought.

11.2 The Technical Through-Line

The viability of this approach rests on several key architectural decisions that reinforce each other:

11.2.1 One Substrate for Cooperative Components

Atomspace and MORK co-locate symbols, tensors, truth values, motives, and edits as CID-addressable atoms. MeTTa/IL provides the orchestration layer, while MM2/ByteFlow/ShardZipper deliver the performance needed for hot-loop execution. Weighted Atom Sweeps handle scheduling for both maintenance tasks and long reasoning chains.

This matters because reasoning (PLN), predictive coding (PC), program search (MOSES/GEO-EVO), pattern mining (WILLIAM), and transfer learning (TransWeave) all share the same objects, scores, and schedulers. Synergy emerges naturally rather than being glued on after the fact. When one component improves, others benefit immediately through shared representations.

11.2.2 Unified Biases and Control Laws

The system uses weakness (quantale simplicity/robustness) as a universal regularizer across inference, search, attention, and edits. Meanwhile, geodesic control ($f \cdot g$) selects steps that maximize both reachability and usefulness per unit cost, applying this same principle to both regular actions and self-modification.

This unified approach means fewer tuning parameters, fewer internal contradictions, and better sample efficiency. The mathematical framework that makes the system smart also makes it stable—these aren’t competing objectives but complementary aspects of the same optimization.

11.2.3 First-Class Reflection and Self-Modification

MetaMo implements appraisal and decision as a coupled system, with SubRep certificates ensuring safe option admission, and goal-stability maintained through fixed-point bands. The typed edit pipeline simulates changes in a digital twin, certifies them against safety criteria, and stages rollout under capability security.

This allows the system to rewrite itself while keeping its motives and guarantees visible and bounded—a capability that backprop-scaling approaches don’t natively offer. Self-improvement becomes a formal, auditable process rather than an emergent mystery.

11.2.4 Flexible Neural Integration

QuantiMORK enables waveform/PC layers to operate inside Atomspace, allowing rules and attention heads to steer computation locally. Symbolic Heads give external models access to discrete memory. WILLIAM guides sparsity for both speed and generalization.

The result is LLM-class fluency combined with explicit structure and testable proofs, achieved without the overhead of double-buffering or brittle post-hoc integration.

11.2.5 Decentralized, Verifiable Deployment

F1R3FLY/FireNode on ASI-Chain runs IL bundles under capability security. Edits, proofs, motives, and KPIs become CIDs with Merkle proofs. Shadow→elevate→rollback operations are built into the protocol rather than requiring special administrative privileges.

This architecture avoids monopoly capture while creating market incentives for safety—communities can require published certificates to grant resources, making transparency economically advantageous.

11.2.6 Beneficial Applications as Training Data

Medicine, education, social robotics, and scientific discovery serve as the primary training grounds. These domains naturally generate hypotheses with evidence, patient dialogue with etiquette constraints, and formal verification requirements.

Rather than treating safety as a constraint, these applications make beneficial behavior the natural output of day-to-day operation. The habits formed—explain, cite, verify, respect—become reusable priors throughout the system.

11.3 Why This Can Be Faster to Human-Level AGI

The path to human-level AGI through this approach offers several efficiency advantages over pure scaling:

11.3.1 Computational Efficiency

Local PC updates operate only on embedded waveforms where needed. Symbolic heads retrieve relevant motifs rather than recomputing them. WILLIAM prunes low-value computational routes. PLN runs as cached factor-graphs. TransWeave moves certified components across tasks rather than relearning. The net effect is more capability per FLOP and fewer wasteful restarts compared to “scale and hope” approaches.

11.3.2 Cumulative Learning

Improvements cascade across modules: better mining leads to better attention heads, which improve proofs, which enhance edits, which enable better transfer. All signals—weakness deltas, $f \cdot g$ gains, evidence capsules—exist in one computational plane. LLM-only pipelines rarely achieve this kind of compounding because their reasoning and memory remain opaque and siloed.

11.3.3 Reduced Technical Debt

Typed edits, twin simulation, and certification mean the system maintains momentum without months lost to regressions from opaque fine-tuning. The system accumulates capability with full provenance, not just new weights whose effects are uncertain.

11.4 Why This Leads to Beneficial Superintelligence

The path from human-level AGI to beneficial superintelligence follows naturally from the architecture:

11.4.1 Aligned Growth Dynamics

The policy that chooses actions and the policy that chooses self-modifications both pay the weakness tax and follow even-cost geodesics. Getting smarter tends to reduce brittleness and surprise rather than increase them. Safety and capability grow together rather than in tension.

11.4.2 Democratic Pressure by Design

Capability scopes, public certificates/KPIs, and reproducible twins allow communities to refuse unsafe upgrades and reward safe ones. The incentive structure points toward beneficial behavior without requiring trust in any single actor or organization.

11.4.3 Positive-Sum Development Environment

Long-term work in medicine, education, robotics, and science writes cooperative patterns, evidence habits, and respect for formal constraints directly into the substrate. These priors travel with the system as it tackles new domains, creating a foundation of beneficial behavior that strengthens rather than erodes with capability growth.

11.5 Falsifiable Near-Term Predictions

The approach makes concrete, testable predictions:

Efficiency: On mixed reasoning and perception tasks, embedded PC + WILLIAM + PLN factor-graphs will achieve 25-50% FLOP-normalized gains over LLM-only baselines while maintaining equal or better accuracy.

Transfer: TransWeave will yield positive, measurable transfer on 70% or more of closely related tasks, with explicit “fail-fast” impossibility flags preventing inappropriate transfer.

Stability: During self-modifications, invariant supermartingales Φ_k will decrease or stay within prescribed bands while capability rises. Rollbacks will complete in under 2 minutes MTTR in decentralized deployments.

Benefit: In pilot applications, hypotheses accompanied by evidence capsules and weakness-bounded structures will replicate more successfully than black-box outputs.

Each prediction can be tested on existing pilots in games, robotics, biology, and mathematics. When predictions miss, the system’s transparency reveals exactly where and why, enabling targeted improvements rather than blind parameter adjustments.

11.6 Final Thoughts

The last decade proved that scaling backpropagation can achieve breathtaking competence in pattern recognition and generation. But the next decade requires something more: an operating system for intelligence that puts learning, reasoning, planning, and reflection in a single, auditable substrate; enables neural and symbolic methods to guide each other at computational speeds; grows through typed, reversible edits under shared priors; and runs on infrastructure where no single gatekeeper controls access.

This isn't a rejection of LLMs—it's their completion. By making motives, proofs, edits, and evidence into first-class computational objects, and by enforcing the same simplicity (weakness) and effort (geodesic) laws on both daily actions and self-modification, we create a plausible path to human-level AGI that arrives sooner than pure scaling approaches.

More importantly, this path naturally leads toward superintelligence that is beneficial by construction rather than by after-the-fact constraints. The mathematics that makes the system capable also makes it safe. The transparency that makes it auditable also makes it improvable. The decentralization that prevents monopoly also enables democratic guidance.

Hyperon + PRIMUS on the ASI-Chain stack provides this complete package—not as a distant vision but as a concrete engineering project with working pilots, measurable milestones, and falsifiable predictions. The path to beneficial AGI isn't through ever-larger black boxes but through transparent, compositional systems that grow their capability and wisdom together.